

Digital Contracting Service

Deployment Guide

As of 2026-08-05

DCS Deployment Guide

This guide brings a DCS instance into operation and keeps it there. It covers administrator documentation items 1 (installation and deployment), 2 (configuration and RBAC) and 5 (upgrade and migration) from SRS Å2.6.

Two neighbouring documents complement it and are not repeated here:

- **Backup and restore:** `docs/backup-integration-guide.md`. Backup inventory, procedures and restore order live there, including the interaction between retention and key destruction.
- **Architecture and runtime behaviour:** the technical manual under `docs/technical-manual/`. Why a component exists, how a flow runs through the system and what a signal means is explained there. This guide says what to do.

Concrete values, commands and configuration references live exclusively in this document. Where another document names a value, the one here applies.

The guide has two parts. **Part A** is the contractual deliverable: operation on Kubernetes. **Part B** describes the local development and test setup and is not relevant for operations.

Part A: Operations

A1 Target environment and prerequisites

A1.1 Delivery form

DCS is delivered exclusively as Linux container images for Kubernetes (DCS-OE-02). The deployment configuration is a Helm chart (DCS-OE-03), delivery into a cluster happens through ArgoCD (section A9), build and publication through GitHub Actions (section A10).

Two images make up the application:

Image	Dockerfile	Contents
DCS backend	<code>deployment/docker/Dockerfile</code>	Backend binary, built frontend, <code>gendid</code> , <code>hsmsign</code> , <code>pacmonitor</code> , SoftHSM2/OpenSC/OpenSSL, the provisioning scripts
pdf-core	<code>pdf-core/Dockerfile</code>	Deterministic PDF/A-3a compiler and verifier including C2PA provenance. The generated documents declare conformance level A of the PDF/A-3 family (ISO 19005-3), so they carry a tagged structure tree and the embedded machine-readable contract representation in the same document

All remaining components (PostgreSQL, Hydra, NATS, ORCE, DSS, IPFS, Fuseki, Keycloak, Federated Catalogue) come from the chart's subcharts with third-party images pinned there.

Two properties of this delivery matter for the procurement and operations decision:

- **DSS is a community image.** No EU-published container image exists for the DSS demo web application; the subchart therefore pulls a community image, pinned to a fixed version, never `latest`. Anyone who has to vouch for the provenance of their images has two options: rebuild the image from the EU source and override `dss.image.repository / .tag`, or run their own validation service and point the instance at it via `dss.url` (A5.6).
- **Fuseki and the Federated Catalogue are bundled convenience.** They hang off the chart as dependencies so that an instance starts "with batteries included" and works completely without external prerequisites. They are not mandated for operations. If an organization already runs a Federated Catalogue, `fcservice.enabled` stays off and the instance is pointed at the existing one via `federatedCatalogue.*` (A5.5).

A1.2 Cluster prerequisites

Prerequisite	Why
Kubernetes cluster with an ingress controller	The <code>/well-known</code> documents, the UI and the API must be reachable externally: peers resolve the instance through its federation documents, wallets call the API directly during the signing ceremony, and target systems report back through the API. The bundled example values for the issuers and <code>sharedServices</code> use <code>nginx.ingress.kubernetes.io/*</code> annotations and therefore assume ingress-nginx ; under a different controller the rewrite rules have to be replaced.
Target namespace already exists	<code>deploy.sh</code> does not create it. That is intentional: the script works with credentials that cannot see cluster-wide objects.
StorageClass for PVCs	Postgres, IPFS, the HSM token, ORCE, Fuseki and fc-postgres each claim one volume. For encryption at rest (DCS-NFR-SEC-14), set an encrypted class per environment, see A5.4.
RBAC: permission to create roles / rolebindings in the namespace	The HSM provisioning hook and the ACME renewal bring their own ServiceAccount/Role/RoleBinding. If that is not permitted, there is an operating mode without API access (A6.5).
Resource envelope	A complete instance is around ten workloads and ten services. Under a <code>ResourceQuota</code> , check <code>limits.cpu</code> in particular; under a <code>LimitRange</code> , the per-container minimum. Both produce failure modes that do not look like failures (A13).
TLS certificate for the public host	A peer resolves this instance via <code>https://<host>/well-known/did.json</code> . <code>cert-manager</code> is the normal case; for clusters where <code>cert-manager</code> cannot schedule the HTTP-01 solver, the chart ships a <code>CronJob</code> fallback (<code>acmeRenewal</code> , see A5.6).

A1.3 Tools on the workstation

`helm` and `kubectl` must be on the `PATH`, otherwise `deploy.sh` aborts. `curl` is needed for the external checks. `docker` only if images are built locally.

Chart version and application version are in `deployment/helm/Chart.yaml` (currently `version: 0.2.2`, `appVersion: "1.16.0"`). `appVersion` is the fallback for `image.tag` and `pdfCore.image.tag` when nothing is set there.

A2 Obtaining and building images

A2.1 Pulling from the Harbor registry

The release workflow `.github/workflows/publish.yml` builds both images and publishes the chart. The registry address is the GitHub Actions secret `HARBOR_HOST`, the credentials are `HARBOR_CREDENTIALS` (Base64 of a JSON object `{username: password}`, not a plaintext password). Project and user are in `deployment/harbor.config`:

```
PROJECT=dcs
USER=dcs
```

Harbor expects the robot account form `robot$<user>`, not the bare `USER` value from this file; `harborconfig.sh` from `eclipse-xfsc/dev-ops` handles the derivation.

The chart is pushed as an OCI artifact to `oci://<HARBOR_HOST>/dcs/charts`. An operator therefore obtains it via:

```
helm pull oci://<HARBOR_HOST>/dcs/charts/digital-contracting-service --version <chart-version>
```

For pulling the images from a private registry there are two routes: reference an existing pull secret via `imagePullSecrets`, or let the chart create one (`dockerRegistry.create=true` with `server`, `username`, `password`; the secret name is `dockerRegistry.secretName`, default `docker-registry`). The second variant writes the credentials into a value set. It therefore belongs in a `secret-manager` invocation via `--set` and not in a values file.

A2.2 Local build

Prerequisites: Docker, access to the repository root directory (both Dockerfiles need it as build context).

Steps:

1. Build the DCS backend image:

```
DOCKER_REGISTRY=<registry> DOCKER_REPO=<repo> bash deployment/docker/build-image.sh <tag>
```

Without `DOCKER_REGISTRY` and `DOCKER_REPO` the script builds `digital-contracting-service:<tag>` locally and does not push. With both variables it builds `<registry>/<repo>/digital-contracting-service:<tag>`, additionally tags `:latest` and pushes both.

2. Build the pdf-core image:

```
docker build -f pdf-core/Dockerfile -t <registry>/<repo>/pdf-core:<tag> .
```

Verification: The backend image contains all four binaries and the provisioning scripts. Observable signal:

```
docker run --rm --entrypoint ls <image> -1 /app/dcs /app/gendid /app/hsm sign /app/pacmonitor /app/scripts/hsm-provision.sh
/app/scripts/c2pa-cert-provision.sh
```

All six paths must be printed. If `/app/gendid` is missing, the HSM provisioning hook cannot generate a `did.json` and the instance never gets through startup; if `/app/pacmonitor` is missing, the compliance CronJob fails on every run.

Failure modes:

Symptom	Cause	Fix
Image starts, <code>dlopen</code> of <code>libsoftasm2.so</code> fails	A build with <code>CGO_ENABLED=0</code> . The PKCS#11 layer is cgo-only.	Use only the bundled Dockerfile; it builds with <code>CGO_ENABLED=1</code> on a glibc toolchain.

The Goa code generator runs hermetically in the build (`GOPROXY=off`) from the module cache. For this the Dockerfile pre-downloads the complete module set (`go mod download all`), including the generator's own dependencies. A build needs no hand-holding.

A2.3 Tag discipline

`image.tag` and `pdfCore.image.tag` must be pinned to an explicit release version in production, never empty and never `latest`. Two reasons, both actually observed:

- An empty tag falls back to `Chart.AppVersion`; that is a chart property, not a deployment decision.
- With HSM provisioning enabled, the hook job uses the same image as the application by default. If the application runs with `pullPolicy: Always` but the job runs with `IfNotPresent` on a moving tag, the node cache can provision an older key set than the application expects. The application then aborts at startup on a key the provisioner never created. That is why `pkcs11.provisioning.image.pullPolicy` is empty by default and follows `image.pullPolicy`.

A3 Chart layout and subchart dependencies

The parent chart is called `digital-contracting-service` and lives under `deployment/helm`. It renders the backend deployment, the pdf-core deployment, services, ingress, the provisioning hooks and the CronJobs. Everything else comes from subcharts.

Subchart	Alias	Enabled by	Role
postgresql	no alias	postgresql.enabled	Bundled database server
hydra	no alias	hydra.enabled	OAuth2/OIDC for browser and machine login
nats	no alias	nats.enabled	Event bus
orce	no alias	orce.enabled	Node-RED flows: trust PDP, archive notary, TSA, contract target, audit executor
dss	no alias	dss.enabled	EU DSS signature validation
federated-catalogue	fcservice	fcservice.enabled	Federated Catalogue service
keycloak	fcKeycloakApp	fcservice.enabled	Identity provider only for the Federated Catalogue
fuseki	fcFuseki	fcservice.enabled	Graph store of the Federated Catalogue
fc-postgres	fcPostgres	fcservice.enabled	Database for fc-service and its Keycloak
ipfs	ipfs	ipfs.enabled	Kubo node, holds all artifact ciphertexts
ipfs-document-manager	ipfsDocumentManager	ipfsDocumentManager.enabled	Tenant gateway in front of IPFS
statuslist-service	statuslistService	statuslistService.enabled	XFSC status list service. Off by default and without consumers; removal of the subchart is pending (A5.6)
kube-prometheus-stack	no alias	monitoring.enabled	Prometheus/Grafana/Alertmanager
traefik	no alias	traefik.enabled	Ingress controller, test clusters only

The four `fc*` subcharts hang together off `fcservice.enabled`. An instance using a **remote** Federated Catalogue sets only `federatedCatalogue.enabled` instead. That is the DCS backend's configuration block for outbound calls, independent of where the catalogue runs.

A3.1 Building dependencies

```
helm dependency build --skip-refresh deployment/helm
```

Run this on every deployment, not just the first. Helm prefers a packed `charts/*.tgz` over the source directory; an archive from an earlier build otherwise shadows changes to `charts/<name>/` and ships old templates. `deploy.sh` does this on its own.

`kube-prometheus-stack` and `traefik` come from external repositories. If they are not known yet:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add traefik https://traefik.github.io/charts
```

`--skip-refresh` builds from the committed `Chart.lock`. Without the two repositories the build fails on the external dependencies.

If `monitoring.enabled` is set, the Prometheus stack CRDs must be created beforehand. They are not part of the release:

```
helm show crds deployment/helm/charts/kube-prometheus-stack-*.tgz | kubectl apply --server-side -f -
```

A4 Environment matrix of the values files

The chart's own `values.yaml` is always applied first. Every file below is layered on top. Order matters: later `--values` override earlier ones.

File	Environment	Assumes	Must be overridden for the target cluster
<code>values.yaml</code>	Base, always active	nothing	Nothing directly; it is deliberately not deployable (no registry, no host, no identity)
<code>values.prod.yaml</code>	Production (skeleton)	External PostgreSQL, real HSM, pre-created secrets	<code>image.repository / tag</code> , all hostnames, <code>pkcs11.modulePath / tokenLabel</code> , <code>signing.issuerX5ChainPath</code> , <code>federatedCatalogue.apiUrl</code> , <code>fcKeycloak.realmURL</code> , <code>identity / DID source</code> , <code>oid4vp.trust</code> , storage classes. See A4.1
<code>values.acceptance.yaml</code>	Acceptance/staging (skeleton)	Pre-created secrets; SoftHSM2 in the cluster	Same as production, except PKCS#11 (stays SoftHSM2). See A4.1
<code>values.dev.yaml</code>	Local development, instance A	Rancher Desktop or similar with NodePort forwarding to <code>localhost</code>	Not suitable for cluster operation: <code>replicaCount: 0</code> , backend and pdf-core run natively
<code>values.dev2.yaml</code>	Local development, instance B	as above, second port range	as above
<code>values.bdd.yaml</code>	kind test cluster, instance A	kind with <code>hostPort 18080</code> , locally loaded images	Not for operations: <code>image.pullPolicy: Never</code> , <code>DCS_ALLOW_DEV_TRUST=true</code> , Traefik with <code>hostPort</code>
<code>values.bdd2.yaml</code>	kind test cluster, instance B	Instance A already runs in the same namespace	as above
<code>values.issuer-base.yaml</code>	Base of every credential issuer release	Always layered as the first <code>--values</code> file, one stack file on top	Nothing directly; it carries the flow set, the volume, the disabled local TSA and the admin endpoints, but no address
<code>values.issuer.yaml</code>	DEMO , credential issuer next to instance A	<code>values.issuer-base.yaml</code> underneath; separate Helm release of the <code>orce</code> subchart	<code>ingress.hosts[].host</code> , <code>ingress.className</code> , <code>ingress.tls</code>
<code>values.issuer2.yaml</code>	DEMO , credential issuer next to instance B	as above	as above
<code>values.issuer.dev.yaml</code>	Credential issuer in the local dev stack (<code>dev-stack.sh</code>)	<code>values.issuer-base.yaml</code> underneath; NodePort forwarding to <code>localhost:30181</code>	Not for operations: <code>pkiRootCA.devFixture</code> hands the issuer the root that lives in the repository, private key included
<code>values.issuer.bdd.yaml</code>	Credential issuer in the kind test cluster	<code>values.issuer-base.yaml</code> underneath; Traefik including middleware CRDs from the BDD release	as above
<code>values.pid-issuer.yaml</code>	DEMO , PID issuer, once per federation	Separate Helm release of the <code>orce</code> subchart	<code>ingress.hosts[].host</code> , <code>ingress.className</code> , <code>ingress.tls</code>

The issuer files marked **DEMO** start a demonstration issuer. It mints credentials without verifying anyone. That holds for the power of attorney as much as for the personal identity proof. Such issuers belong in a demo, an acceptance run or a test setup. They do not belong anywhere near a contract with legal effect. What a production operation needs instead is in A7. The dev and BDD overlays start the same demo issuer from the same base; they differ only in how it is reached and in the fact that it receives the committed fixture root so that the equally committed trust anchors of the test stack match it.

A4.1 `values.prod.yml` and `values.acceptance.yml` are skeletons

Both files describe the **pattern** of an environment. They are not a deployable configuration. They carry placeholders (`dcx.example.org` , `[registry]` , empty `apiURL`) and leave several mandatory values open. The render passes with them; the failures only show up in the pod, in this order:

1. **`values.prod.yml` : the pod never leaves bootstrap.** `pkcs11.modulePath` and `pkcs11.tokenLabel` are empty and `pkcs11.provisioning.enabled=false` . Without real values the backend finds no token and permanently logs `waiting for PKCS#11 token` .
2. **The pod exits with `Could not read did document`.** `identity.enabled` is not set in either file. Without it, the deployment renders no `DCS_DID` .
3. **The pod exits while loading the auth configuration.** `oid4vp.trust.enabled` is not set in either file and therefore `false` ; no `OID4VP_TRUST_DATA_PATH` is rendered, and the backend aborts with `oid4vp configuration missing` . Setting `oid4vp.trust.enabled=true` while leaving `existingConfigMap` empty already makes the chart refuse the render with a pointer to the dev fixture (A7.4).
4. **Catalogue login fails.** Both files set `federatedCatalogue.oauth.clientID: "dcx-fc-client"` (as does the chart default). The realm imported by the bundled Keycloak does not create this client; there it is called `federated-catalogue` . Against the bundled catalogue the token request therefore answers with 401. Anyone running an external Keycloak sets the client actually created there.
5. **`values.prod.yml` additionally: no certificate chain.** With the provisioning hook disabled, nobody issues the chain the instance needs for its own status list and the C2PA signature. It must be mounted into the backend container and its path set in `signing.issuerX5ChainPath` (A6.3); otherwise the backend exits with a pointer to `DCS_ISSUER_X5CHAIN_PATH` .

Both files additionally carry a trap: they set `ipfs.mfsBaseURLSecretRef` , but **no template reads this value**. `IPFS_MFS_BASE_URL` is produced exclusively from `ipfsClient.mfsBaseURL` or, with the `ipfs` subchart enabled, from the in-cluster derivation. Anyone who disables the bundled IPFS and relies on this secret reference starts an instance whose IPFS configuration is empty.

This is how the files are meant. Hostnames, registry, identity and secret wiring are deployment state and belong in a values file **outside this repository**. Treat the two files as a checklist rather than as configuration.

A4.2 Environment-dependent values

The following values are only correct under a specific cluster variant and must be checked on any switch:

Value	Valid under	Effect elsewhere
<code>traefik.ports.web.hostPort: 18080</code> (<code>values.bdd.yml</code>)	kind with a matching <code>extraPortMappings</code> configuration	On k3s, Klipper's <code>svclb</code> already occupies the port; the Traefik pod stays <code>Pending</code>
<code>ingress.className: traefik</code> (<code>values.bdd.yml</code>)	Test clusters with the bundled Traefik	In production, set the class of the existing controller
<code>nginx.ingress.kubernetes.io/*</code> annotations (issuer values, <code>sharedServices</code>)	<code>ingress-nginx</code>	Without effect under Traefik or Istio; the path rewrites must be replaced
Ordering of <code>/.well-known/did.json</code> vs. Hydra's <code>/.well-known</code>	<code>ingress-nginx</code> prefers the more exact path	Under controllers without this preference, list the three documents as full paths as in <code>values.bdd.yml</code>
<code>image.pullPolicy: Never</code> (BDD values)	kind with <code>kind load docker-image</code>	In any other cluster no pod starts
empty <code>storageClassName</code>	Clusters with a sensible default class	Without a default the claim stays unbound and the pod <code>Pending</code>
<code>pkcs11.provisioning.persistence.size: 1Gi</code>	CSI drivers with a minimum size	The token needs around 10 MiB; 1 Gi is there because several drivers do not provision below it
<code>service.type: NodePort</code> with fixed ports (dev values)	Rancher Desktop	In a shared cluster the ports collide

Enforcing the TLS version

`DCS-IR-CI-01` and `DCS-NFR-SEC-01` require every connection to a DCS interface to run over TLS 1.3 and explicitly rule out SSL 3.0 as well as TLS 1.0 and 1.1. A TLS certificate alone does not achieve this: which protocol versions get negotiated is decided by the ingress controller, and the defaults of the common controllers still allow TLS 1.2.

If Traefik from this chart terminates TLS, `ingress.tlsOptions.enabled: true` creates a `TLSOption` with `minVersion: VersionTLS13` and points the ingress at it via annotation. The two belong together; the resource without the annotation has no effect, which is why the chart sets the annotation itself.

```

ingress:
  tlsOptions:
    enabled: true
    minVersion: "VersionTLS13"

```

The option is off by default because it creates a Traefik CRD. In a cluster without this CRD the installation would otherwise fail.

If the platform's ingress terminates TLS (`values.prod.yml` leaves `ingress.className` empty and thereby takes the cluster's default class), the chart cannot enforce anything. The setting then belongs in the controller:

Controller	Setting
ingress-nginx	<code>ssl-protocols: "TLSv1.3"</code> in the controller's ConfigMap. The annotation <code>nginx.ingress.kubernetes.io/ssl-protocols</code> only works when the controller runs with <code>--enable-ssl-passthrough</code> ; the ConfigMap route is the reliable one.
Traefik (platform installation)	<code>TLSoption</code> in the controller's namespace, referenced via <code>traefik.ingress.kubernetes.io/router.tls.options</code>
Istio gateway	<code>spec.servers[].tls.minProtocolVersion: TLSV1_3</code>
Cloud load balancer in front	The provider's TLS policy; with an upstream terminator, that terminator decides, not the cluster

Measure instead of assuming. The requirement is only met when an older version is actually rejected. Two calls against the public address suffice as evidence:

```

# must succeed
openssl s_client -connect dcs.example.org:443 -tls1_3 </dev/null 2>&1 | grep -E "Protocol|Cipher"

# must fail
openssl s_client -connect dcs.example.org:443 -tls1_2 </dev/null 2>&1 | grep -qE "handshake failure|no protocols available" \
  && echo "TLS 1.2 is refused" || echo "ERROR: TLS 1.2 is still accepted"

```

The second call belongs in the acceptance run. Without it, the only thing proven is that TLS 1.3 is possible, not that older versions are ruled out.

A5 Configuration reference by effect

Listed here are only values for which a rendering location in the chart **and** a reading location in the code are both established. A value that can be set but is read by nothing is not listed. It would fake a knob that does nothing.

A5.1 Identity, routing and public address

Value	Rendered effect	Effect in the system
<code>route.basePath</code>	<code>DCS_API_PATH = <basePath>/api</code> , <code>DCS_UI_PATH = <basePath>/ui</code> , ingress path	The prefix under which API and UI live. Peers and callbacks derive their URLs from it.
<code>paths.api</code> , <code>paths.ui</code>	Override the two derived paths individually	Only needed when API and UI do not live under the same prefix
<code>route.publicBaseURL</code>	<code>DCS_PUBLIC_BASE_URL</code> (verbatim) and <code>DCS_PUBLIC_URL</code> (scheme + <code>didHostname</code> + path)	Both are mandatory at startup. <code>DCS_PUBLIC_URL</code> is the base of every absolute IRI in generated documents; <code>DCS_PUBLIC_BASE_URL</code> the public API base for wallet round trips.
<code>route.didHostname</code>	Host part of the <code>did:web</code> identifier and the anchor URL	Only set this when the <code>did:web</code> hostname is not the host from <code>publicBaseURL</code> . Empty means: host from <code>publicBaseURL</code> , otherwise <code>localhost:<service.port></code> .
<code>signing.issuerDID</code> or <code>signing.issuerDIDSecretRef</code>	<code>ISSUER_DID</code>	The DID of this instance. The provisioning hook uses it verbatim for <code>did.json</code> ; only when it is empty does it derive one from the hostname. This is the way to run several instances under one hostname with path segments.
<code>identity.enabled</code>	Renders <code>DCS_DID</code> and mounts the identity source	Mandatory for every in-cluster deployment. Without it no <code>DCS_DID</code> , without <code>DCS_DID</code> no DID document, without a DID document no startup.
<code>identity.secretName</code> , <code>identity.mountPath</code>	Name of the secret and mount path	Default: <code><fullname>-identity</code> under <code>/app/identity</code> . The name is derived from the release so that two releases in one namespace do not collide.
<code>ingress.enabled</code> , <code>.className</code> , <code>.hosts</code> , <code>.tls</code> , <code>.annotations</code> , <code>.pathType</code>	Ingress object	<code>ingress.enabled=true</code> without at least one entry in <code>hosts</code> aborts the render.
<code>service.port</code>	Container port, service port, ingress backend port	Default 8991

A5.2 Data storage and messaging

Value	Rendered effect	Effect in the system
<code>database.secretRef.name / .key</code>	<code>DATABASE_URL</code> from a secret	Preferred route. When set, it wins over all other <code>database.*</code> fields.
<code>database.url</code>	<code>DATABASE_URL</code> as a literal	Only without <code>secretRef</code>
<code>database.port/user/password/name/sslmode</code>	Components of the derived DSN	Only apply when neither <code>secretRef</code> nor <code>url</code> is set and a Postgres host is resolvable
<code>postgresql.enabled</code>	Bundled server; the host becomes <code><release>-postgresql</code>	Without it, <code>serviceDiscovery.postgresqlHost</code> or a complete DSN must be set
<code>postgresql.additionalDatabases</code>	Additional databases at first startup	Every component with its own DSN on this server needs an entry here. Hydra only runs <code>hydra migrate sql</code> and creates no database. Without an entry, Hydra stays not ready forever.
<code>postgresql.persistence.enabled</code>	PVC instead of <code>emptyDir</code>	Default is <code>emptyDir</code> . With that, a pod restart loses every contract. Turn it on for anything beyond a throwaway cluster.
<code>messaging.secretRef / messaging.natsURL / messaging.natsPort</code>	<code>NATS_URL</code>	Without <code>nats.enabled</code> and without <code>serviceDiscovery.natsHost</code> the URL stays empty
<code>ipfs.enabled</code> , <code>ipfs.persistence.*</code>	Kubo node and its volume	The volume holds all pinned artifact ciphertexts. Losing it is an audit trail loss, not merely a document loss.
<code>ipfs.tenantBaseURLSecretRef</code>	<code>IPFS_TENANT_BASE_URL</code> from a secret	The way to source the tenant address from a secret instead of the values file
<code>ipfsClient.tenantBaseURL</code> , <code>ipfsClient.tenantName</code>	<code>IPFS_TENANT_BASE_URL</code> explicitly, or the tenant path	Without values, derived as <code>http://<release>-ipfs-document-manager:8080/v1/tenants/<tenantName></code>
<code>ipfsClient.mfsBaseURL</code>	<code>IPFS_MFS_BASE_URL</code>	Without a value, derived as <code>http://<release>-ipfs:5001</code>

Both IPFS variables are mandatory at startup; if they are missing, the backend aborts with `IPFS configuration missing` . The `ipfs.*` values configure the **bundled** Kubo node. Anyone attaching an existing IPFS node instead sets the addresses via `ipfsClient.tenantBaseURL` and `ipfsClient.mfsBaseURL` . The environment variables are produced from these two fields. A secret reference exists only for the tenant address; an `ipfs.mfsBaseURLSecretRef` can be declared but is read by no template (A4.1).

A5.3 Authentication, roles and machine identities

Value	Rendered effect	Effect in the system
hydra.enabled	Hydra subchart	Without Hydra no browser login and no machine tokens
hydra.config.selfIssuerURL	HYDRA_PUBLIC_ISSUER_URL	The address Hydra stamps into every redirect URL. Must be externally reachable and match the ingress host.
hydra.config.internalIssuerURL	HYDRA_INTERNAL_ISSUER_URL	Address through which the backend reaches Hydra inside the cluster. Empty means <code>http://<release>-hydra:<publicPort></code> .
hydra.config.loginURL , .consentURL , .logoutURL	Hydra's redirect targets	Must point at the DCS UI and its callback paths
hydra.config.dev	Hydra dev mode	Only permissible when <code>selfIssuerURL</code> is <code>http://</code> . In production <code>false</code> .
hydra.clients[0]	HYDRA_CLIENT_ID , HYDRA_CLIENT_SECRET , HYDRA_REDIRECT_URI , HYDRA_POST_LOGOUT_REDIRECT_URI , plus HYDRA_ADMIN_URL	Only the first entry is rendered into the backend. It is the browser client. Further entries exist in Hydra, but the backend does not know them as its own login client.
hydra.route.paths	Ingress rules towards Hydra	Default <code>/oauth2</code> and <code>/.well-known</code> . The second entry is the reason the three DCS <code>/.well-known</code> documents are routed separately and exactly.
hydra.secrets.system	Hydra's system secret	There is no <code>secretKeyRef</code> equivalent in this chart version. Inject the value via <code>--set</code> from the secret manager, never write it into a values file.
systemClients	DCS_SYSTEM_CLIENTS (JSON array)	Seed for the machine identities. Every entry needs <code>client_id</code> , <code>participant_id</code> and <code>roles</code> . The roles come from the configuration, never from the token; a client cannot request more than it has been given. An unknown role name aborts startup.

Valid role names (exact spelling; any other value stops startup):

Human roles: `Template Creator` , `Template Reviewer` , `Template Approver` , `Template Manager` , `Contract Creator` , `Contract Reviewer` , `Contract Approver` , `Contract Manager` , `Contract Negotiator` , `Contract Signer` , `Contract Observer` , `Archive Manager` , `Auditor` , `Sys. Administrator` , `Compliance Officer` , `Integration Manager` .

System roles: `Sys. Contract Creator` , `Sys. Contract Reviewer` , `Sys. Contract Approver` , `Sys. Contract Manager` , `Sys. Contract Signer` , `Sys. Auditor` , `Contract Target System` .

Three properties of this list matter operationally:

- **Sys. Contract Signer carries no signing permission.** The class exists so that the refusal is provable. A signature always requires a natural person with a wallet.
- **System roles alone are not enough for the catalogue.** The three catalogue endpoints (`/catalogue/template/retrieve` , `/catalogue/template/retrieve/{did}` , `/catalogue/template/search`) require one of the roles `Template Manager` , `Contract Creator` , `Contract Reviewer` , `Contract Approver` , `Contract Manager` or `Contract Signer` . All six are human roles. A machine token carrying only system roles receives `insufficient permissions` there. The check is a pure name match over the role list and does not distinguish role classes: if a machine client is assigned one of the named human roles, via `systemClients` or via the machine identity management, it is admitted to the catalogue. Neither the startup check nor the management refuses that. Anyone who wants to give a machine client catalogue access therefore does so deliberately and owns the consequence: a catalogue entry pulled by a machine is attributable to no human.
- **Contract Target System is not assigned via systemClients** but through the registration of a target system (see `contractTargets`).

An entry in `systemClients` alone is not enough: the same `client_id` must also exist under `hydra.clients` , otherwise the client has nowhere to authenticate. Conversely, a client that exists only under `hydra.clients` authenticates successfully and is then rejected everywhere.

`systemClients` is a **seed**, not a runtime source. The entries are copied into the machine identity registry at startup so that an instance has its callers before anyone logs in. After that, machine identities are administered in the management UI.

A5.4 Signing material and PKCS#11

Value	Rendered effect	Effect in the system
<code>pkcs11.modulePath</code>	<code>PKCS11_MODULE_PATH</code>	Path to the PKCS#11 module inside the container. Default <code>/usr/lib/softhsm/libsofthsm2.so</code> .
<code>pkcs11.tokenLabel</code>	<code>PKCS11_TOKEN_LABEL</code>	Default <code>dcS</code>
<code>pkcs11.pinSecretRef.name / .key</code>	<code>PKCS11_PIN</code> from a secret	If <code>name</code> is empty, the chart creates a secret <code><fullname>-hsm-pin</code> from <code>pkcs11.pin</code> . For production, always reference a pre-created secret.
<code>pkcs11.pin</code>	Content of the automatically created secret	Ignored as soon as <code>pinSecretRef.name</code> is set
<code>pkcs11.provisioning.enabled</code>	HSM PVC, provisioning hook, <code>SOFTHSM2_CONF</code> , token mount	<code>true</code> co-deploys a SoftHSM2 software token. That is dev/staging/CI, never a production HSM.
<code>pkcs11.provisioning.tokenDir</code>	Mount path of the token volume and value of <code>SOFTHSM2_CONF</code>	Default <code>/var/lib/softhsm</code>
<code>pkcs11.provisioning.soPin</code>	SO PIN at token initialization	Used only at first initialization
<code>pkcs11.provisioning.publishSecrets</code>	Whether the hook writes secrets through the Kubernetes API	See A6.5
<code>pkcs11.provisioning.persistence.size/accessModes/storageClassName</code>	The token PVC	<code>storageClassName</code> is where the encryption of the key directory is decided (DCS-NFR-SEC-14)
<code>pkcs11.provisioning.resources</code>	Resource block of the hook job	Set this on namespaces with a <code>limits.cpu</code> quota
<code>pkcs11.provisioning.image.repository/tag/pullPolicy</code>	Image of the hook job	Empty means: the same image as the application
<code>signing.tsaURL</code>	<code>TSA_URL</code>	Mandatory at startup. Empty aborts. See A5.8 for what this URL points at by default.
<code>signing.tsaTrustCertSecret</code>	<code>TSA_TRUST_CERT_FILE = /etc/dcs/tsa-trust/tsa-cert.pem</code> , plus a secret mount	Replaces the TSA certificate embedded in the binary. The secret must carry the key <code>tsa-cert.pem</code> . Switching timestamp authorities is thus a configuration change without a rebuild on the backend side.
<code>signing.issuerX5ChainPath</code>	<code>DCS_ISSUER_X5CHAIN_PATH</code>	Path of the PEM chain publishing this instance's <code>dcS-c2pa</code> key. It signs the C2PA manifests and the self-served status list (A5.6). Empty means: the chain the provisioning hook drops on the token volume. With <code>pkcs11.provisioning.enabled=false</code> it is mandatory, otherwise the backend does not start.

pdf-core holds no key material and no certificate: the backend signs the COSE Sig_structures itself with the `dcS-c2pa` token key and sends the corresponding chain along with every request in the `X-DCS-C2PA-X5Chain` header. The chart therefore configures neither a key nor a secret for pdf-core.

Three storage classes carry material at rest and must be set to an encrypted class per environment: `postgresql.persistence.storageClassName` (contracts and wrapped CEKs), `ipfs.persistence.storageClassName` (all artifact ciphertexts) and `pkcs11.provisioning.persistence.storageClassName` (the private SoftHSM2 keys). After the deployment, `deploy.sh` checks whether the bound claim actually carries this class. A PVC keeps the class it was created with; a later values change has no effect.

A5.5 Federation and trust

This section has three value groups: the trust gate towards peers, the trust document for presented credentials and the connection to the Federated Catalogue. The tables name the value and the rendered effect; what a value means in operation follows as prose below each table.

Trust gate towards peers

Value	Rendered effect
<code>federation.trustPdpURL</code>	<code>DCS_TRUST_PDP_URL</code> , only when set

The trust gate is the policy endpoint against which every inbound and outbound federated interaction is checked. It is **fail-closed**: as long as the URL is empty or the endpoint does not answer with 2xx, the instance sends nothing and accepts nothing. It does not report this on its own, so the outage looks like silence. The check for it is in A12.2. Every instance consults exclusively its own endpoint; there is no shared gate across a federation.

Trust document for presented credentials

Value	Rendered effect
<code>oid4vp.trust.enabled</code>	Renders <code>OID4VP_TRUST_DATA_PATH</code>
<code>oid4vp.trust.existingConfigMap</code>	ConfigMap name of the trust document
<code>oid4vp.trust.existingConfigMapKey</code>	Key inside it
<code>oid4vp.trust.existingConfigMapMountPath</code>	Mount path inside the container
<code>oid4vp.trust.dataPath</code>	Path without a ConfigMap
<code>oid4vp.trust.x5cAnchorsPath</code>	<code>OID4VP_X5C_TRUST_ANCHORS_PATH</code>

Without `oid4vp.trust.enabled` no `OID4VP_TRUST_DATA_PATH` is rendered, and the backend exits at startup with `oid4vp configuration missing`; a trust document is a startup gate rather than an option (A12.1). The intended route is your own ConfigMap. It is not managed by Helm and therefore survives an uninstall of the release; an instance's trust decisions are thus not tied to the lifecycle of its deployment. If `existingConfigMap` stays empty, `dataPath` applies and points at the dev fixture shipped in the image. Why that triggers a render abort and where the exception lies is in A7.4.

`x5cAnchorsPath` names the trust anchors against which a credential is checked when it brings its keys as an x5c chain. If the path stays empty, such a credential is rejected. The instance never believes the leaf certificate the credential itself delivers.

Against the same anchors the instance also checks signed status lists that bring their keys as an x5c chain in the token header; the leaf certificate must identify the issuer the token names. An unsigned status list is accepted in no configuration (ADR-34); a switch permitting it does not exist in the backend. The anchor bundle must therefore contain the roots of **all** issuers whose credentials or status lists the instance verifies, not only the PID issuer's (A7.2). One anchor the instance adds itself: the root of its own chain from `signing.issuerX5chainPath`, because it verifies the self-served status list (A5.6) through the same path as any foreign one, and this root only comes into existence at provisioning time.

The content structure of the trust document (purposes, organizations, mechanisms) is in A7.3.

Connection to the Federated Catalogue

Value	Rendered effect
<code>federatedCatalogue.enabled</code>	The entire FC configuration block
<code>federatedCatalogue.apiURL</code> or <code>.secretRef</code>	<code>FEDERATED_CATALOGUE_API_URL</code>
<code>federatedCatalogue.oauth.clientID</code>	<code>FEDERATED_CATALOGUE_CLIENT_ID</code>
<code>federatedCatalogue.oauth.clientSecretSecretRef</code> or <code>.clientSecret</code>	<code>FEDERATED_CATALOGUE_CLIENT_SECRET</code>
<code>fcKeycloak.realmURL</code>	<code>FC_KEYCLOAK_REALM_URL</code>
<code>federatedCatalogue.remote.acknowledgeAdminAllTrustBoundary</code>	Render approval with a remote catalogue

The catalogue block is **all-or-nothing**: as soon as one of the four values API URL, client ID, client secret and Keycloak realm URL is set, the backend demands all four at startup. If one is missing, the abort message names it.

Three values are the usual sources of error:

- **`apiURL` must contain the servlet context path of the fc-service.** A public URL usually carries it already, an in-cluster address does not. Without it the catalogue answers every call with 404 (A13.3).
- **`oauth.clientID` must be a client the realm in use actually creates.** The bundled realm creates `federated-catalogue`; the chart default `dcs-fc-client` does not match it and gets 401 against the bundled catalogue.
- **`fcKeycloak.realmURL` is the public realm URL.** fc-service checks the token's `iss` claim against it; a token minted through an in-cluster address is rejected. The bundled realm is called `federated-catalogue-realm` and lives under the base `/auth`; the URL thus has the form `https://<host>/auth/realms/federated-catalogue-realm`.

`acknowledgeAdminAllTrustBoundary` is mandatory `true` when `federatedCatalogue.enabled` is set without `fcservice.enabled`, that is, against a remote catalogue. The service access to the catalogue carries `ADMIN_ALL`. Every instance on that catalogue can therefore manage every entry in it, including those of the others. A shared catalogue is therefore only permissible within a jointly trusting administrative boundary, and the acknowledgement forces someone to take that decision consciously.

Bundled catalogue

Value	Rendered effect
<code>fcservice.enabled</code>	Catalogue, Keycloak, Fuseki and fc-postgres in the release
<code>fcservice.realmProvision.enabled</code>	Realm import hook
<code>fcservice.keycloak.hostname</code>	Issuer URL that fc-service expects
<code>fcKeycloakApp.hostname</code>	Public hostname of the Keycloak
<code>fcservice.portal.enabled</code>	Demo portal of the catalogue
<code>fcservice.route.path</code>	Ingress rule to <code>fc-service:8081</code>
<code>fcKeycloakApp.auth.adminUser / .adminPassword</code>	Admin access of the bundled Keycloak, as a secret
<code>fcservice.realmProvision.adminUser / .adminPassword</code>	Access with which the import hook creates the realm

These values concern only the bundled catalogue (A1.1). The realm import is required with a fresh catalogue database; it runs through the Keycloak admin API. `fcservice.keycloak.hostname` and `fcKeycloakApp.hostname` must carry the same public hostname. That is manual work, because a values file cannot reference itself. If `fcservice.keycloak.hostname` stays empty, fc-service builds the issuer URL from the in-cluster service; behind a real ingress that is wrong and leads to the 401 patterns from A13.3. `fcservice.route.path` publishes the catalogue through the ingress so that an instance from another cluster can reach it.

`fcservice.portal.enabled` toggles the catalogue's web UI. It is **on** by default and serves demonstration purposes. DCS does not need it, it talks to the catalogue through its API. Where resources are tight or the published surface should stay small, it can be off.

Both Keycloak admin accounts carry `admin / admin` in the chart defaults and must be overridden before any reachable deployment. One is the administrator of the bundled Keycloak, the other the access with which the import hook creates the realm. Both must carry the same value, otherwise the import fails at login. Set the value via `--set` from the secret manager, not in a committed values file. The import hook renders it as plaintext into its job specification; anyone allowed to `get job` in the namespace can read it there, because a `secretKeyRef` equivalent does not exist for this value in this chart version. Read permissions on jobs in the namespace are therefore to be kept as tight as read permissions on secrets.

A5.6 Dependent services and callbacks

Value	Rendered effect	Effect in the system
<code>dss.enabled</code> or <code>dss.url</code>	<code>DSS_URL</code>	The validator for externally produced signatures. Without it the instance rejects every signature; contract creation keeps working, the outage only shows at signing time. If <code>DSS_URL</code> is set but the instance cannot reach it, validation is a hard failure.
no chart value, only via <code>extraEnv</code>	<code>STATUSLIST_LIST_ID</code>	The instance's own list in which new revocation entries are allocated; the default is list 1. It is only set once list 1 is full, and only after the operator has registered the successor list in the <code>status_list_cursors</code> table of the <code>dcx</code> database. With an unregistered list, allocation fails hard instead of issuing an entry nobody serves. A non-positive value aborts startup.
<code>pdfCore.enabled</code> or <code>pdfCore.url</code>	<code>PDF_CORE_URL</code>	Mandatory at startup
<code>pdfCore.contextIRI</code>	<code>PDF_CORE_CONTEXT_IRI</code>	The JSON-LD <code>@context</code> IRI the backend embeds in every contract and template envelope. Empty means: <code><pdf-core-url>/ontology/dcs-pdf-core</code> .
<code>pdfCore.ontologyBaseURL</code>	<code>DCS_PDF_CORE_ONTOLOGY_BASE_URL</code> in the pdf-core container	Base of the ontology IRIs in payloads and in the served JSON-LD context. Empty means: the in-cluster service URL. Behind a public address, set that address; it must match the <code>contextIRI</code> .
<code>pdfCore.replicaCount</code> , <code>.resources</code> , <code>.service.port</code>	pdf-core deployment	Sizing of the renderer. It is the bottleneck for bulk generation and for re-rendering incoming documents for cross-checking
<code>signing.tsaURL</code>	<code>TSA_URL</code>	See A5.4 for the effect and A5.8 for what this URL points at by default
<code>archiveNotary.baseURL</code> , <code>.notaryPath</code> , <code>.auditLogPath</code>	<code>ORCE_ARCHIVE_NOTARY_URL</code> , <code>ORCE_ARCHIVE_AUDIT_LOG_URL</code>	Empty <code>baseURL</code> plus <code>orce.enabled</code> yields <code>http://<release>-orce:<orce-port></code>
<code>global.archiveAuditTokenSecretRef</code> or <code>global.archiveAuditToken</code>	<code>ORCE_ARCHIVE_AUDIT_LOG_BEARER_TOKEN</code>	Without the token the archive integrity check fails
<code>auditExecutor.url</code> , <code>.path</code> , <code>.timeout</code>	<code>PAC_AUDIT_EXECUTOR_URL</code> , <code>PAC_AUDIT_EXECUTOR_TIMEOUT</code>	Empty <code>url</code> plus <code>orce.enabled</code> yields the bundled ORCE endpoint. A customer replaces the executor purely through configuration.
<code>global.auditExecutorTokenSecretRef</code> or <code>.auditExecutorToken</code>	<code>PAC_AUDIT_EXECUTOR_BEARER_TOKEN</code>	Only rendered when one of the two is set
<code>workflowGateExecutor.url</code> , <code>.path</code> , <code>.timeout</code>	<code>PAC_WORKFLOW_GATE_EXECUTOR_URL</code> , <code>PAC_WORKFLOW_GATE_EXECUTOR_TIMEOUT</code>	An invalid timeout value aborts startup
<code>global.workflowGateTokenSecretRef</code> or <code>.workflowGateToken</code>	<code>PAC_WORKFLOW_GATE_EXECUTOR_BEARER_TOKEN</code>	Only rendered when one of the two is set. Without the token the gate call is not authorized
<code>orce.checkpointSink.enabled</code> , <code>.url</code> , <code>.timeoutSeconds</code> , <code>.internalTimeoutSeconds</code> , <code>.bearerToken</code> or <code>.bearerTokenSecretRef</code> , <code>.allowInsecureTLS</code>	<code>CHECKPOINT_SINK_*</code> in the ORCE container	Publishes audit checkpoints in order to an external HTTPS sink. When enabled, the chart refuses the render without an HTTPS URL, without a positive internal timeout or without a bearer token. A <code>sequence_gap</code> or <code>previous_root_mismatch</code> reported by the sink blocks publication permanently and is to be treated as an integrity incident; the procedure is in <code>docs/audit-checkpoint-workflow-gate-operations.md</code> . <code>allowInsecureTLS</code> only works together with the test controls and stays off in production.
<code>contractTargets</code>	ConfigMap plus <code>CONTRACT_TARGETS_FILE = /etc/dcs/contract-targets/targets.json</code>	Initial content of the target system registry. Seeding is creating, not reconciling , matched by <code>name</code> : an entry re-pointed in the UI is not reset at the next startup, and an entry removed here does not disappear from a running instance.
<code>sharedServices.enabled</code> , <code>.host</code> , <code>.services[]</code> , <code>.ingressClassName</code> , <code>.tls</code> , <code>.proxyBodySize</code> , <code>.annotations</code>	Ingress under <code>https://<host>/shared/<name></code>	Publishes this instance's stateless services for a peer that cannot run its own. This reduces that peer's independence and is a deliberate decision.
<code>acmeRenewal.enabled</code> , <code>.host</code> , <code>.email</code> , <code>.secretName</code> , <code>.server</code> , <code>.schedule</code> , <code>.image</code> , <code>.challengePort</code> , <code>.rbac.create</code> ,	CronJob, service, ingress and optionally RBAC for an HTTP-01 renewal	Fallback for clusters where cert-manager cannot schedule the solver pod. <code>host</code> , <code>email</code> and

Value	Rendered effect	Effect in the system
<code>.serviceName</code> , <code>.ingressClassName</code> , <code>.ingressAnnotations</code> , <code>.resources</code> , <code>.persistence</code>		<code>secretName</code> are mandatory, otherwise the render aborts. Where cert-manager works, leave this off.
<code>complianceMonitor.enabled</code> , <code>.schedule</code> , <code>.startingDeadlineSeconds</code> , <code>.timeout</code> , <code>.resources</code>	CronJob <code><fullname>-pac-monitor</code>	Runs the compliance pass on a schedule instead of on request. Default every five minutes with <code>concurrencyPolicy: Forbid</code> . Switched off, risks are only found when someone asks.

The instance's own status list (ADR-34)

Every issuer signs and serves its status list itself, and an unsigned list is accepted nowhere. For the credentials the instance itself mints, that is, the contract lifecycle credential in every generated PDF and the signature summary next to it, this means: the instance is the issuer and publishes their revocation status itself.

- **Address.** The list lives at `GET /status-list/<n>` on the instance's **origin root**, like `did.json`, not under the API prefix. A verifier only has the URL the credential names and must not have to know this instance's API prefix. The chart's ingress therefore routes `/status-list/` as its own prefix rule to the backend. The origin is derived from `route.publicBaseURL` and `route.didHostname` and is at the same time the `iss` of the served token.
- **Signature.** The list is a `statuslist+jwt`, signed with the `dcsc2pa` token key; the chain from `DCS_ISSUER_X5CHAIN_PATH` sits as `x5c` in the header. Its leaf certificate must carry the origin and the instance's `did:web` as SANs, otherwise every verifier rejects the list as coming from an unidentified issuer. The provisioning hook stamps in both identifiers (A6.2).
- **Data.** Allocation and revocation bits live in the `dcsc` database (`status_list_cursors` and the revocation tables); nothing is reported to a separate service. The revocation data therefore belongs to the normal database backup per `docs/backup-integration-guide.md`.

The subchart of the XFSC status list service can be deployed via `statuslistService.enabled` but has **no consumer**: the backend knows no `STATUSLIST_SERVICE_URL`. The dev and BDD stacks run it until the subchart is removed; a deployment that turns it on runs a service that nothing reads.

A5.7 Platform and operational mechanics

Value	Rendered effect	Effect in the system
<code>replicaCount</code>	Backend replicas	<code>autoscaling.enabled</code> overrides the field
<code>autoscaling.enabled</code> , <code>.minReplicas</code> , <code>.maxReplicas</code> , <code>.targetCPUUtilizationPercentage</code>	HorizontalPodAutoscaler	When enabled, the HPA replaces <code>replicaCount</code> as the source of the replica count
<code>readinessProbe</code>	Readiness probe on <code>/readyz</code>	Deliberately separated from process liveness. See A12.1.
<code>livenessProbe</code>	Liveness probe on the backend container as soon as the value is set	Not set by default , so the backend container has no liveness probe. That matches the bootstrap behaviour from A12.1: a pod waiting minutes for its token is not dead. Anyone who wants one sets one here that tolerates this wait.
<code>resources</code>	Resource block of the backend container	Without the block, on namespaces with a LimitRange the default there applies
<code>podSecurityContext</code> , <code>securityContext</code> , <code>nodeSelector</code> , <code>tolerations</code> , <code>affinity</code> , <code>podAnnotations</code> , <code>podLabels</code> , <code>hostAliases</code>	Taken straight into the deployment	The usual placement and hardening knobs; the chart does not interpret them, it passes them through
<code>volumes</code> , <code>volumeMounts</code>	Additional volumes on the backend	The way to mount, for example, the PID root CA for <code>oid4vp.trust.x5cAnchorsPath</code>
<code>extraEnv</code> , <code>extraEnvFrom</code>	Additional environment variables	<code>extraEnvFrom</code> also disables the render-time check of the dev fixture, because its content is not readable at render time
<code>customCA.enabled</code> , <code>.configMapName</code>	Mount to <code>/usr/local/share/ca-certificates/custom</code>	For TLS connections against your own CA
<code>serviceAccount.create</code> , <code>.name</code> , <code>.annotations</code> , <code>.automount</code>	ServiceAccount of the backend	With FC realm provisioning active, the <code>-fc-lifecycle</code> account is used instead, because the backend then waits for the hook job
<code>monitoring.enabled</code> and the blocks <code>grafana</code> , <code>prometheus</code> , <code>alertmanager</code> , <code>nodeExporter</code> , <code>kubeStateMetrics</code>	kube-prometheus-stack as a subchart	Only needed when the instance is supposed to bring its own monitoring. The bundled example values use <code>storageClassName: local-path</code> ; in a cluster without this class the PVCs stay unbound. Grafana is additionally off by default; anyone enabling it overrides <code>grafana.adminPassword</code> , because a placeholder sits there.
<code>istio.enabled</code> , <code>.hosts</code> , <code>.gateways</code>	VirtualService instead of an ingress object	For clusters where ingress runs through a service mesh gateway. <code>hosts</code> and <code>gateways</code> must match the existing gateway objects; the chart creates none.

A5.8 The timestamp authority is local by default

`TSA_URL` is a startup gate. The obvious assumption that the instance talks to an external timestamp authority does **not** hold in the default configuration.

The ORCE subchart sets `localTSA.enabled: true`. The `/tsa` flow signs timestamp responses inside the pod itself with a certificate that the `localTSA.autoProvision` hook generates and stores in the secret `<release>-orce-tsa-material` (keys `tsa-cert.pem`, `tsa-key.pem`). No traffic leaves the cluster, and the serial number counter lives at `/data/tsa-state/serial` on the ORCE volume.

Operational consequences:

- A timestamp from the default configuration is a **development timestamp**. It proves nothing towards third parties. Anyone who needs RFC 3161 evidence with probative value switches to a real authority.
- The switch is a pure configuration change on the backend side and requires **no rebuild of the backend**. The certificate embedded in the binary is only the default value, `TSA_TRUST_CERT_FILE` overrides it.
- If `localTSA` is disabled, the automatically provided secret goes away too. If `signing.tsaTrustCertSecret` still points at it, the mount fails.

The path to a trustworthy timestamp

For attaching a real authority there are two routes. Both end at the same two values on the backend: `signing.tsaURL` pointing at the authority and `signing.tsaTrustCertSecret` pointing at its CA certificate (key `tsa-cert.pem`).

- Direct.** `signing.tsaURL` points at the authority's RFC 3161 endpoint. That is the shortest route. It fits when the instance is allowed to reach the authority itself.
- Through ORCE.** ORCE is a Node-RED-based low-code environment, and the `/tsa` endpoint is a flow inside it. This flow can be replaced by one that forwards the timestamp request to a commercial authority, DigiCert for example, and returns its response. `signing.tsaURL` then stays pointed at ORCE. **This is the intended path from the development timestamp to a trustworthy one.** Credentials, quotas and a fallback target can be maintained there without a code change, and the instances of a federation keep seeing the same endpoint address.

Such a forwarding flow is **not** part of the delivery. The chart ships only the local development flow. The operator creates the forwarding flow in ORCE themselves and disables `localTSA.enabled` in the process, so that signing no longer happens in the pod.

Verification that the timestamp no longer originates locally: The certificate the backend verifies timestamps against comes from `signing.tsaTrustCertSecret`. After the switch, this secret must carry the CA certificate of the external authority and must no longer be `<release>-orce-tsa-material`:

```
kubectl -n <ns> get deployment/<release>-digital-contracting-service \
-o jsonpath='{.spec.template.spec.volumes[?(@.name=="tsa-trust")].secret.secretName}'
```

A newly generated contract then carries a timestamp whose issuer name names the external authority. As long as the ORCE secret name is still there, the instance keeps verifying against the self-signed development material.

A6 Identities, secrets and HSM

A6.1 The instance's keys

Every private key of a DCS instance lives in a PKCS#11 token (DCS-IR-HI-01). There is no software fallback. Five keys, all ECDSA P-256:

Label	Purpose	Notes
<code>dc-did</code>	Signs this instance's identity proofs (JAdES, DCS-to-DCS challenge)	Must match the published <code>did.json</code> , otherwise startup aborts
<code>dc-vc</code>	Signs lifecycle and federation credentials	None
<code>dc-oid4vp-jar</code>	Signs OpenID4VP request objects	None
<code>dc-c2pa</code>	Signs C2PA claims and the self-served status list (A5.6)	The backend sends its x5chain (<code>DCS_ISSUER_X5CHAIN_PATH</code>) to pdf-core with every request for the COSE header; the same chain sits as x5c in the status list token
<code>dc-ecdh</code>	Wraps content encryption keys	Needs <code>CKA_DERIVE</code> . At startup a wrap/unwrap self-test runs against the <code>keyAgreement</code> entry in <code>did.json</code> ; if it fails, the instance does not start.

The labels can be overridden via `DCS_HSM_KEY_DID`, `DCS_HSM_KEY_VC`, `DCS_HSM_KEY_JAR`, `DCS_HSM_KEY_C2PA` and `DCS_HSM_KEY_ECDH` (through `extraEnv`).

There is no contract signing key. A contract signature is created in the signing person's wallet. DCS verifies it.

The reason is legal, not technical. Article 26 of the eIDAS regulation (EU) 910/2014 requires an advanced electronic signature to be uniquely linked to the signatory, capable of identifying the signatory, and created using signature creation data that the signatory can, with a high level of confidence, use under their sole control. A key in the DCS's HSM, with which the instance signs for all signatories, does not satisfy sole control. The derivation is in ADR-12; ADR-1 excludes the contract signature from the list of DCS keys for the same reason.

A6.2 SoftHSM2 in the cluster (dev, staging, CI)

With `pkcs11.provisioning.enabled=true` the chart renders:

- a PVC `<fullname>-hsm-token`,
- a secret `<fullname>-hsm-pin` (unless a `pinSecretRef` is set),
- a `post-install,post-upgrade` hook job `<fullname>-hsm-provision`.

The job initializes the token, generates the five keys, issues the C2PA x5chain against a dev CA living in the token directory, and derives `did.json` from the `dc-did` key with `/app/gendid`. Into the leaf certificate of the chain it stamps the status list origin and the instance's `did:web` as SANs plus a CRL distribution address; without these identifiers the self-served status list would be rejected by every verifier (A5.6). The chain remains on the token volume as `c2pa-x5chain.pem`, where the backend reads it via `DCS_ISSUER_X5CHAIN_PATH`. With `publishSecrets=true` the job additionally publishes `did.json` as the identity secret. It is idempotent and runs on every install and upgrade.

SoftHSM2 is a software token and **not a production HSM**.

A6.3 Real HSM (production)

Careful, this is not pure configuration in every case. On the backend side, module path, token label and PIN are values. The vendor module itself, however, has to get into the container, and depending on the vendor an additional volume is not enough for that: if the module brings its own libraries, a configuration file at a fixed location, a client daemon or a license file, a **rebuild of the container image** containing these parts is required. On top of that come possible **hardware-specific changes to the deployment**: access to a PCIe or USB device through a device plugin,

a `hostPath` or extended container privileges, a network path to the network HSM, a dedicated init container for client registration. Therefore plan the switch to a vendor HSM as a change to image and deployment, not as a values change. Which parts are needed exactly is in the HSM vendor's integration documentation; the DCS repository contains no vendor-specific template.

Prerequisites:

1. The vendor's PKCS#11 module is reachable inside the backend container, through an additional volume from `volumes / volumeMounts` or through a derived image (see the warning above).
2. All five keys already exist in the token, `dcS-ecdh` with `CKA_DERIVE`. The chart creates no HSM keys in production.
3. A secret with the token PIN exists in the target namespace.
4. The PEM certificate chain publishing the HSM's `dcS-c2pa` key is mounted into the **backend container** (through `volumes / volumeMounts`, usually from a pre-created secret) and its path set in `signing.issuerX5ChainPath`. Without the provisioning hook, nobody issues this chain. Its leaf certificate must identify the instance as issuer: the status list origin (`https://<didHostname>`) and the `did:web` as SAN entries, because verifiers, its own included, reject a status list whose leaf does not name the issuer (A5.6). The chain must end in a CA certificate.
5. A `did.json` publishing exactly the token keys exists as a secret and is referenced via `identity.secretName`. It must carry a `keyAgreement` entry for `dcS-ecdh`.

Values:

```
pkcs11:
  modulePath: /usr/lib/<vendor>/libpkcs11.so
  tokenLabel: <token-label>
  pinSecretRef:
    name: <pin-secret>
    key: PKCS11_PIN
  provisioning:
    enabled: false

signing:
  issuerX5ChainPath: <mount-pfad>/x5chain.pem

identity:
  enabled: true
  secretName: <did-secret>
```

Verification: After the rollout, no line `waiting for PKCS#11 token` may appear in the backend log anymore, and `HTTP server listening` must follow:

```
kubectl -n <ns> logs deployment/<release>-digital-contracting-service --tail=200 | grep -E "waiting for PKCS#11 token|HTTP server listening"
```

Failure modes: see A13.

A6.4 Secret inventory

No secret value appears in this document. Listed are origin, consumers and the effect of absence.

Secret	Origin	Consumers	Effect when missing
<fullname>-hsm-pin (or pkcs11.pinSecretRef)	Chart from pkcs11.pin, or pre-created	Backend, HSM provisioning job	No token access, backend stays in bootstrap
<fullname>-identity	HSM provisioning hook (only with publishSecrets=true)	Backend as DCS_DID	Backend reads no DID document and exits
Certificate chain for signing.issuerX5ChainPath (any form, usually a pre-created secret via volumes)	Operator, issued from the real HSM's dcs-c2pa key	Backend (DCS_ISSUER_X5CHAIN_PATH)	Only relevant with pkcs11.provisioning.enabled=false; the backend exits at startup (A6.3)
database.secretRef	Operator or secret manager	Backend, pac-monitor CronJob	No database connection
messaging.secretRef	Operator	Backend	No event bus
federatedCatalogue.secretRef, federatedCatalogue.oauth.clientSecretSecretRef	Operator	Backend	Catalogue login fails; startup aborts
ipfs.tenantBaseURLSecretRef	Operator	Backend	IPFS configuration incomplete, startup aborts
global.archiveAuditTokenSecretRef	Operator	Backend	Archive integrity check impossible
global.auditExecutorTokenSecretRef, global.workflowGateTokenSecretRef	Operator	Backend	The respective executor call is not authorized
orce.checkpointSink.bearerTokenSecretRef (or .bearerToken)	Operator	ORCE	With the checkpoint sink enabled, the chart refuses the render without a token (A5.6)
signing.tsaTrustCertSecret (key tsa-cert.pem)	Operator or ORCE TSA hook	Backend	The TSA certificate embedded in the binary applies
<release>-orce-tsa-material (keys tsa-cert.pem, tsa-key.pem)	ORCE subchart hook (localTSA.autoProvision) or pre-created	ORCE	The local TSA flow does not sign
Hydra system secret and client secrets	Operator via --set	Hydra	This chart version has no secretKeyRef for these
fc-keycloak (keys admin-user, admin-password)	Chart from fcKeycloakApp.auth.*	Bundled Keycloak	No admin access to the catalogue Keycloak. The chart default is admin / admin and must be replaced before any reachable deployment (A5.5). The name is fixed and not derived from the release: two releases in the same namespace collide on it
imagePullSecrets or dockerRegistry.*	Operator	All pods	Images are not pulled

There is no internal service credential beyond the roles from A5.3: the background PDF regeneration signs in-process through the PKCS#11 key and calls no authenticated route of its own.

A6.5 Operating without RBAC permissions

If the installing identity may not create roles / rolebindings in the namespace, the install fails with cannot create resource roles. Then:

```
pkcs11:
  provisioning:
    publishSecrets: false
orce:
  localTSA:
    autoProvision: false
acmeRenewal:
  rbac:
    create: false
    serviceAccountName: <bestehender-account>
```

Consequences that come with this:

- The provisioning hook writes nothing through the Kubernetes API. did.json and c2pa-x5chain.pem stay on the token volume, which the backend pod mounts anyway; the backend reads DCS_DID and the chain from there. pdf-core needs no mount in any mode, it receives the chain from the backend with every request (A5.4).

- The TSA material secret must be created beforehand. It is self-signed and depends on nothing in the cluster.

A7 Credential issuers and the trust document

The bundled issuer is a DEMO credential issuer. DCS does not ship a real credential issuer.

That holds for both kinds. The bundled power-of-attorney issuer issues a credential with the display name "FACIS Demo Power of Attorney" without checking the authority. The bundled PID issuer issues an identity proof without checking the person; its credential is not a EUDI PID and must not be presented as one. Both generate their key material themselves at first start and are attached to no trust infrastructure.

For operation with legal effect, the operator provides the issuers themselves: the power-of-attorney issuer from their own authorization system, the identity proof from a publisher that actually verifies persons. Both bind to an instance through the trust document (A7.3). DCS does not prescribe which products these are; it only requires that they speak OID4VCI and are verifiable through one of the mechanisms named in A7.3.

A `dc`s release contains **no** credential issuer. The two credentials a signing person presents are the power of attorney and the personal identity proof (PID). Both come from OID4VCI issuers running as separate Helm releases of the `once` subchart, each with its own volume, its own key and its own DID.

Release	Values files	Issues	Count
<code>dc</code> s-issuer	<code>values.issuer-base.yml</code> + <code>values.issuer.yml</code>	Demo power of attorney	One per DCS instance, under its hostname
<code>dc</code> s-issuer	<code>values.issuer-base.yml</code> + <code>values.issuer2.yml</code>	Demo power of attorney	For the second instance, in its namespace
<code>dc</code> s-pid-issuer	<code>values.pid-issuer.yml</code>	Demo PID	Once for the entire federation

The power-of-attorney issuer is always installed from two files: `values.issuer-base.yml` carries everything that is the same in every stack (flow set, volume, disabled local TSA, admin endpoints), the second file only what the target cluster does differently. Dev stack and BDD suite install the same issuer from the same base under the same release name; a stack that deviates from the production form cannot test its behaviour.

`flowsDir` selects a release's flow set: `flows` is the DCS-side set (trust PDP, archive, TSA, contract target), `flows-issuer` the power-of-attorney issuer, `flows-pid-issuer` the PID issuer. A release runs exactly one set, because the sets claim overlapping routes.

Every issuer also serves the status list of its own credentials (ADR-34): under its path prefix as `GET /status-list/1`, signed with its own key as a `statuslist+jwt` with an embedded x5c chain, plus an `/admin` flow through which a revocation is set. The revocation bits live on the issuer volume and survive a pod restart. The issuers need no separate status list service.

Two properties of the base file an operator must know before publishing the issuer:

- **The admin endpoints carry no authorization of their own.** `values.issuer-base.yml` enables `adminFlows.enabled` so that a revocation remains settable; `POST /admin/credentials/<idx>/revoke` and `/unrevoke` are then part of the same published prefix as `/offer`. Anyone who makes the issuer reachable publishes these two endpoints along with it and puts an authenticating proxy in front.
- **The status list must answer at exactly the URI the credentials name.** The issuer builds the `sub` of the served token from the incoming host and the announced path prefix (`X-Forwarded-Prefix` or `ISSUER_PATH_PREFIX`), and the verifier rejects a list whose `sub` does not match the URI from the credential. Hence exactly one host rule per issuer, and hence an ingress that strips the prefix must announce it at the same time.

The PID issuer is a third party towards **every** instance. An instance that itself issues the identity proof it later accepts as evidence about its signatories has attested nothing. With the power-of-attorney issuer it is the other way round: it speaks *for* an organization about who may sign for it, and therefore belongs to it.

A7.1 What the operator has to supply

The repository delivers the shape of an issuer. It does not deliver an identity:

- **Own hostnames.** Every host in the example values is a placeholder, as are the ingress class and the empty `tls` list. An issuer is published under the hostname of the instance it serves, so that its DID resolves as `did:web:<host>:issuer`.
- **Own keys and DIDs.** Every issuer generates root CA and signing key into its volume at first start and publishes them under its path prefix as `pki/root-ca.pem`, `pki/jwks.json` and `.well-known/did.json`.
- **A dedicated trust document per DCS instance.** Nothing derives it: it names issuers by identifiers that only exist once the issuers run.

A7.2 Order and commands

Prerequisites: Namespaces exist; ingress class and TLS are settled for the target hostnames; the DCS releases are **not** yet installed or run without a trust document.

Steps:

1. Install an issuer next to each DCS instance, always with the base file first:

```
helm upgrade --install dcs-issuer ./deployment/helm/charts/orce \
  -n <namespace> \
  -f ./deployment/helm/values.issuer-base.yml \
  -f ./deployment/helm/values.issuer.yml
```

2. Install the PID issuer once:

```
helm install dcs-pid-issuer ./deployment/helm/charts/orce \
  -n <namespace> -f ./deployment/helm/values.pid-issuer.yml
```

3. Read out the published identities:

```
curl https://<host>/issuer/.well-known/did.json
curl https://<host>/issuer/pki/jwks.json
curl https://<host>/issuer/pki/root-ca.pem
curl https://<pid-host>/pid-issuer/pki/root-ca.pem
```

4. Write the trust document per DCS instance (structure in A7.3) and create it as a ConfigMap:

```
kubectl create configmap dcs-oid4vp-trust -n <namespace> --from-file=trust.json=<datei>
```

5. Only then install or update the `dcs` release with:

```
oid4vp:
  trust:
    enabled: true
    existingConfigMap: dcs-oid4vp-trust
    x5cAnchorsPath: /etc/dcs/oid4vp-x5c/root-ca.pem
```

At exactly this path, a PEM bundle with the root CAs of **all** issuers is mounted through `volumes / volumeMounts`, the power-of-attorney issuer's as well as the PID issuer's. The bundle is assembled from what each running issuer actually serves under `pki/root-ca.pem`, and compared by fingerprint, not by subject: the demo issuers mint their roots at first start, all carry the same subject `CN = FACIS Demo Root CA`, and a bundle with only one root rejects the lists and credentials of the others, while every error message names the same CA name. If an issuer mints its root anew, after a deleted volume for instance, the bundle must be rebuilt; until then the check fails closed.

Verification:

```
curl -o /dev/null -w '%(http_code)\n' https://<host>/issuer/.well-known/did.json
kubectl -n <ns> get configmap dcs-oid4vp-trust -o jsonpath='{.data.trust\.json}' | head -c 200
```

The first call must return `200`, the second the beginning of the document. After the rollout the backend log must show no error loading the trust document.

Failure modes:

Symptom	Cause	Fix
Render aborts with a pointer to the dev fixture	<code>oid4vp.trust.enabled=true</code> but no <code>existingConfigMap</code> and no <code>DCS_ALLOW_DEV_TRUST</code>	Create and reference the ConfigMap as above
Pod stays in <code>ContainerCreating</code> , event names a missing ConfigMap	<code>existingConfigMap</code> points at a name that does not exist	Create the ConfigMap. The mount is deliberately not <code>optional</code> : a missing trust document is fatal at startup anyway, and the mount error is easier to diagnose.
Backend exits while loading the trust document	A <code>mechanism</code> value this build cannot resolve	Use only <code>jwtks</code> , <code>x5c</code> , <code>did:jwt</code> , <code>did:web</code> or <code>orcb</code> . The check happens at load time, not at first wallet contact.
Login works, the signing ceremony rejects the power of attorney	Your own issuer lacks the <code>peer</code> purpose	Add the purpose and roll the backend out again
Login or ceremony fails at the status list or x5c check although the trust document is correct and the list is correctly signed	The anchor bundle lacks one issuer's root. All demo roots carry the same subject; the error message therefore names a CA name that appears to be present in the bundle	Reassemble the bundle from the actually served roots and compare by fingerprint (step 5)
Revocation check rejects the status list although it is reachable and correctly signed; the result reads like a revoked credential	The <code>sub</code> of the served token does not match the URI the credential names: the issuer is reached through a different origin or path than the one it was published under, or the ingress strips the prefix without announcing it	Publish the issuer at exactly the URI the credentials name; when stripping the prefix set <code>X-Forwarded-Prefix</code> or pass <code>ISSUER_PATH_PREFIX</code> ; no second host rule on the same release

A7.3 Structure of the trust document

Trust is granted **per purpose** (ADR-31):

Purpose	Meaning
<code>login</code>	May establish a session on this instance
<code>peer</code>	Checked when the credential arrives from a counterparty, above all in the signing ceremony
<code>pid</code>	May attest a natural person

Every issuer appears **twice** in the document: under its `did:web` identifier, whose key is read live from the issuer's DID document, and under the `https` identifier the issuer actually sets in `iss`. The second is not `did:web`-resolvable and therefore carries its key through the chosen mechanism.

```
{
  "vcts": ["urn:dcs:poa:v1", "urn:dcs:pid:demo:v1"],
  "peer_dynamic": false,
  "issuers": {
    "did:web:dcs.example.org:issuer": {
      "purposes": ["login", "peer"],
      "organizations": ["did:web:dcs.example.org"],
      "mechanism": "did:web"
    },
    "https://dcs.example.org/issuer": {
      "purposes": ["login", "peer"],
      "organizations": ["did:web:dcs.example.org"],
      "mechanism": "jwtks",
      "jwtks": { "keys": [ "<der ver ffentlichte JWKS des Issuers>" ] }
    },
    "did:web:pid.example.org:pid-issuer": { "purposes": ["pid"], "mechanism": "x5c" },
    "https://pid.example.org/pid-issuer": { "purposes": ["pid"], "mechanism": "x5c" }
  }
}
```

Rules that follow from the model:

- **organizations limits what an issuer may speak about.** A credential naming an organization not listed there is rejected even with a valid signature. The explicit wildcard `"*"` permits any organization and must be written out. A missing list does not mean "all".
- **An instance grants login and peer to its own issuer.** `peer` is what a signing ceremony checks the local signatories' power of attorney against. Granting it to a foreign issuer would mean a counterparty authorizes itself through a document it publishes.
- **The PID issuer receives exclusively pid.** An identity proof must not grant access.
- **Counterparties are not enumerated.** Whether a counterparty is dealt with at all is decided by the trust gate via `DCS_TRUST_PDP_URL`, dynamically and fail-closed.

- **Revocation is checked for every accepted credential and every purpose** and fails when the list is unreachable. An unsigned status list is never accepted (ADR-34). A list with an x5c chain is checked against the anchors from `x5cAnchorsPath`, one without a chain against its issuer's JWKS stored in the trust document.

The document binds issuer keys. If an issuer key is regenerated, the document silently loses its validity: logins fail without anything having changed in the configuration. **Every regeneration of an issuer key requires an update of every trust document that names it.**

A7.4 The dev fixture

If `existingConfigMap` stays empty, the release falls back to a fixture shipped in the image whose issuer keys live in the repository. Anyone with a copy of the repository can use them to mint credentials that an instance configured this way accepts. The chart therefore refuses the render unless the release additionally declares `DCS_ALLOW_DEV_TRUST=true` via `extraEnv`. This declaration belongs exclusively in a dev or CI stack.

If a release sets `extraEnvFrom`, the render-time check is dropped, because its content is not readable at render time; the content-based check in the backend itself then applies.

A8 Initial deployment

A8.1 Prerequisites

All of the following must be satisfied before the first `deploy.sh` run:

1. Target namespace exists.
2. Images are in a registry reachable from the cluster, tags are pinned, a pull secret exists or is created by the chart.
3. The credential issuers run and the trust document sits as a ConfigMap in the target namespace (A7). Order: **issuers before DCS**.
4. All secrets from A6.4 that this deployment references exist.
5. With a real HSM: token, all five keys and the `did.json` secret exist (A6.3).
6. A dedicated values file outside this repository is written. It carries hostnames, DIDs, registry, secret references and storage classes.
7. The release name is settled. It is not cosmetic: subchart service names derive from it, values such as `http://<release>-orcs:1880` must match it.
8. DNS for the public hostname points at the cluster, TLS is settled.

A8.2 Steps

1. Check values and templates against the cluster without changing anything:

```
./deployment/helm/deploy.sh --values <ihre-werte>.yaml \
    --namespace <ns> --release <release> \
    --dry-run
```

Expected output: Dry run passed: values and templates are valid and the API accepted every manifest.

2. Run the deployment:

```
./deployment/helm/deploy.sh --values <ihre-werte>.yaml \
    --namespace <ns> --release <release> \
    --public-url https://<host>
```

`--values` is repeatable; later files override earlier ones. The chart's own `values.yaml` is always applied first, so only the deviations need to be passed. Further options: `--kubeconfig`, `--timeout` (default 15m), `--skip-verify`.

The script first builds the dependencies from the `Chart.lock`, then runs `helm upgrade --install`, and verifies afterwards. It is convergent: the same invocation covers initial installation and update.

3. The script itself waits for the bundled catalogue (if present) and for the backend rollout, and then runs the checks from A8.3.

Important: **No `helm --wait`**. Helm holds back `post-install` hooks until all deployments are ready. The backend, however, only becomes ready after the HSM hook has generated its material. That is a deadlock. `deploy.sh` breaks it by calling Helm without `--wait` and waiting separately afterwards.

A8.3 Verification

After the rollout, `deploy.sh` checks the following. Each check corresponds to one way in which an instance can look healthy and still be unable to work; all were observed on a running deployment that reported no errors.

Check	Observable signal	What it rules out
Backend startup	HTTP server listening in the log	A pod being Running before it has passed its startup gates, because the key material only arrives after scheduling
Catalogue sync	No failed to sync federated catalogue in the log	The backend treats this as fatal and restarts; all that is visible is a rising restart count
Catalogue readiness	No federated catalogue readiness gate failed in the log	A 404 from a wrongly set context path
Trust gate	DCS_TRUST_PDP_URL set and the endpoint answers 2xx	The gate is fail-closed. Unreachable means: federation completely off, without the instance reporting anything. A reachable URL is not enough, the endpoint must answer.
Validator	DSS_URL set	Without a validator the instance rejects every externally produced signature. Contract creation keeps working, the outage only shows at signing time.
Storage classes	Postgres, IPFS and HSM token PVCs bound to the configured class	An existing claim keeps its class; a later values change has no effect
Federation documents	/.well-known/did.json, /.well-known/dcs-agreement-credential.json and /.well-known/dcs-federation-rules.md return 200 from the outside	Hydra's OIDC discovery makes a second claim on /.well-known and shadows them. Inside the cluster everything looks correct; only an external fetch reveals the 404.
DID match	The served did.json carries the same id as ISSUER_DID	A mismatch makes peers reject the instance, visible only on the other side

The external checks only run with `--public-url`. Without it the script says so explicitly instead of passing silently.

Two further signals worth checking that the script does not cover:

```
# Alle Pods laufen oder sind abgeschlossen
kubectl get pods -n <ns> --no-headers | grep -vE 'Running|Completed'

# Der Login-Einstieg antwortet
curl -o /dev/null -w '%{http_code}\n' -X POST https://<host><basePath>/api/auth/login
```

The first line must print nothing, the second `200`.

About the output Helm shows after the install: with ingress enabled it names the correct URL including `route.basePath`. The ClusterIP branch, however, suggests a port-forward and then names `http://127.0.0.1:8991` **without** the base path. Anyone following that address lands on nothing. The complete local address is `http://127.0.0.1:<lokaler-port><basePath>/ui/`.

A8.4 Failure modes at initial deployment

See A13.

A9 GitOps through ArgoCD

This route has never been tested. Neither in CI nor in an acceptance environment has a DCS release ever been rolled out through ArgoCD. The manifest below is derived from the artifacts and plausible, but unproven. Whoever takes this route is the first. Plan accordingly: try it against a throwaway namespace first, not against the acceptance environment, and expect the ordering of the provisioning hooks (A9.1) to be the point where it jams. The proven route for initial installation and upgrade is `deploy.sh` (A8, A11).

The chart ships an Application manifest: `deployment/argocd/application.yaml`.

```
kubectl apply -n argocd -f deployment/argocd/application.yaml
```

The shipped manifest describes the acceptance environment (`dcs-acceptance`, namespace `dcs-acceptance`, `valueFiles: values.yaml + values.acceptance.yaml`). For production it is duplicated and `valueFiles` as well as `destination` adjusted.

Three spots in the shipped manifest must be replaced before applying it for the first time:

- `repoURL` carries an SSH URL to a specific fork repository. Set it to your own repository, and ArgoCD needs stored credentials for it.
- `targetRevision` is `main`. For operations a fixed revision belongs there, not a moving branch.
- `valueFiles` references `values.acceptance.yaml`, a skeleton. A sync against the unchanged manifest renders, but the application never becomes healthy: trust document and identity are missing, and the pod exits at the startup gates from A4.1 one after the other. Your own values file must be added, and the secrets from it must not live in the GitOps repository.

Set `syncOptions: CreateNamespace=true` and `ServerSideApply=true`. The retry policy is five attempts with exponential backoff from 5s to 3m.

A9.1 Why auto-sync is deliberately off

`syncPolicy.automated` is not set. Four reasons stand behind that, three from the chart and one from operations:

1. **An unattended `selfHeal` can mask an intended action as drift.** A key rotation exercise looks to a reconciler like a deviation from the rendered state.
2. **The FC realm hook must run before the application can become healthy.** Realm provisioning is a `post-install,post-upgrade` hook. A reconciler that maps Helm hooks onto its own phases typically runs `post-*` only once the sync is **healthy**. The backend, however, treats a failed catalogue sync as fatal and enters a restart loop until the hook assigns the roles to its service account. The hook that would unlock the application never runs, because the application never becomes healthy. Plain `helm` does not couple `post-install` hooks to pod health, so there the ordering resolves itself.
3. **Components of the chart patch what the chart renders.** The realm hook writes the FC issuer URL into `fc-service` after the manifests have been applied. A reconciler comparing live state against rendered state sees permanent drift there and works against it.
4. **Provisioning is a first-boot side effect, not declarative state.** Key material, `did.json` and the C2PA x5chain are created once into a volume. A reconciler cannot know that a repeated provisioning run is only harmless because the scripts are idempotent.

The recommended flow: ArgoCD holds the desired state, reports drift and creates the sync task. The sync is triggered manually or through a pipeline approval. For initial installation and for upgrades that touch the provisioning hooks, `deploy.sh` is the more reliable route because it knows the ordering.

`automated.prune` / `selfHeal` can be enabled once the operator has verified the four points for their environment.

Verifying an ArgoCD deployment:

```
kubectl -n argocd get application dcs-acceptance -o jsonpath='{.status.sync.status} {.status.health.status}'
```

Expected: `Synced Healthy`. After that the same substantive checks as in A8.3 apply, above all the external `/.well-known` fetches, which ArgoCD knows nothing about.

A10 CI/CD through GitHub Actions

Workflow	Trigger	Effect
<code>publish.yml</code>	Published release, manual	Builds the DCS image and the pdf-core image via the <code>eclipse-xfsc/dev-ops</code> <code>dockerbuild</code> workflow to Harbor; packs the chart and pushes it as an OCI artifact to <code>oci://<HARBOR_HOST>/dcs/charts</code>
<code>bdd-kind.yml</code>	Pull request, push to <code>main</code> , manual	Three parallel jobs: the complete BDD suite on its own kind cluster, an isolated audit gate, and the Playwright E2E suite on a further kind cluster. Reports are published as check annotations and artifacts.
<code>lint-and-format.yml</code>	see the workflow file	Static checks
<code>build-node-red-package.yml</code>	Published release, pull request, push to <code>main</code> , manual	Builds the Node-RED deployment package from <code>deployment/node-red</code> ; on a release the <code>.tgz</code> is attached to it
<code>sbom.yml</code>	Published release, monthly, manual	Calls the central SBOM workflow
<code>eclipse-dash.yml</code>	Published release, monthly, manual	License scan via Eclipse Dash

The chart publication builds the dependencies explicitly anew because `charts/*.tgz` is not in the repository: the local `file://` subcharts resolve from their source directories, the external dependencies (`kube-prometheus-stack`, `traefik`) have to be fetched.

Verifying a publication:

```
helm show chart oci://<HARBOR_HOST>/dcs/charts/digital-contracting-service --version <version>
```

Must print the expected `version` and `appVersion`.

A11 Upgrade, migration and rollback

A11.1 What happens automatically during an upgrade

Step	Triggered by	Note
Database migrations	Backend at startup	All not yet applied SQL files are applied in alphabetical order and recorded in <code>schema_migrations</code> . There are no backward migrations.
Semantic hub seed and anchor refresh	Backend at startup	If this aborts, the instance does not start
HSM provisioning	<code>post-install, post-upgrade</code> hook	Idempotent: an existing token and existing keys stay untouched
FC realm provisioning	<code>post-install, post-upgrade</code> hook	Idempotent; the backend waits via init containers for the job to be created and to finish
Seeding of the target system registry	Backend at startup	Only creates, never changes or deletes
Seeding of the machine identities	Backend at startup	From <code>DCS_SYSTEM_CLIENTS</code>

A11.2 Upgrade procedure

Prerequisites:

1. A current backup per `docs/backup-integration-guide.md` exists, above all of the `dc`s database, because the wrapped CEKs exist only there.
2. The release notes of the target revision are checked for new mandatory configuration. A new mandatory variable is the most common reason an upgrade silences a deployment that ran before.
3. With a real HSM: if the new revision requires an additional key, it already exists in the token and is published in `did.json`. The chart creates no HSM keys in production.
4. The target image tag is pinned.

Steps:

1. Dry run against the cluster:

```
./deployment/helm/deploy.sh --values <ihre-werte>.yaml \
    --namespace <ns> --release <release> --dry-run
```

2. Save the current values of the running release. With manually adjusted configuration this is the only record:

```
helm -n <ns> get values <release> > <release>-werte-vor-upgrade.yaml
```

3. Run the upgrade (the same invocation as for the initial deployment):

```
./deployment/helm/deploy.sh --values <ihre-werte>.yaml \
    --namespace <ns> --release <release> \
    --public-url https://<host>
```

Verification: The complete checklist from A8.3. Additionally:

```
helm -n <ns> history <release>
kubectl -n <ns> get pods -l app.kubernetes.io/instance=<release> \
    -o 'custom-columns=POD:.metadata.name,READY:.status.containerStatuses[*].ready,RESTARTS:.status.containerStatuses[*].restartCount'
```

The new revision must be `deployed`, and the backend container's restart count must not keep rising after the rollout. A rising restart count with `Running` status is the picture of a fatal startup error.

Failure modes: see A13.

A11.3 Rollback and its limits

```
helm -n <ns> rollback <release> <revision>
```

What a rollback does **not** undo, and what follows from that:

- **Schema migrations.** They are forward-only and have no counterparts. A rollback to an older image against an already migrated database is only permissible if the older revision tolerates the newer schema. If it does not, the route is restore per `docs/backup-integration-guide.md`, not a Helm rollback.
- **Provisioned key material.** Token, `did.json` and `x5chain` live on a volume and stay as they are. That is intended: a rollback must not change the instance's identity.
- **State written by hooks:** the Keycloak realm, the role assignment of the catalogue service account.
- **Registry entries** changed through the UI. The `contractTargets` seeding only creates.

A rollback thus takes back configuration and images, not the data model. This procedure has not been exercised in a running cluster; it is derived from the artifacts.

A11.4 Key rotation

Signing keys are versioned: version 1 is the unsuffixed base label, every rotation puts a `-v<N>` key next to it. Old versions stay in the token so that historical signatures remain verifiable.

```
DATABASE_URL=<dsn> bash scripts/rotate-hsm-key.sh \
  <token-dir> <token-label> <pin> <basis-label> [modul] [ca-dir] [x5chain-out]
```

The script generates the next key pair (for `dcS-ecdh` with `--usage-derive`), optionally issues a leaf certificate against the dev CA and advances the pointer in `pki_active_key_version`. `DATABASE_URL` is mandatory; without it the script aborts.

It is tailored to a SoftHSM2 token. Against a vendor HSM, replace the key generation with the vendor procedure; advancing the pointer in `pki_active_key_version` remains the same step.

The script has one limit at `dcS-c2pa`: the leaf certificate it optionally issues carries **no** SAN entries. It is therefore unusable as the chain for `signing.issuerX5ChainPath`, because the self-served status list is rejected without the issuer identifiers in the leaf (A5.6). The chain for status list operation is issued with `scripts/c2pa-cert-provision.sh`, which stamps in origin and `did:web` as SANs.

Rotating `dcS-ecdh` requires the published `did.json` to carry the new `keyAgreement` entry, otherwise the self-test fails at the next startup. Rotating an issuer key requires updating every trust document that names it (A7.3).

A12 Runtime behaviour and verification

A12.1 What the operator sees

Readiness. The backend claims its listen address immediately and answers `/readyz` with 503 until initialization is complete. That includes opening the PKCS#11 token, the functional catalogue gate and the schema sync. The readiness probe polls `/readyz` every 2 seconds with a 1 second timeout and 3 failure attempts.

A pod in state `Running` with `0/1 Ready` and a log showing `bootstrap HTTP server listening ... (initializing)` is **normal** in the first minutes after an install. It does not have to be an error.

Liveness. The backend container has no liveness probe by default (A5.7). pdf-core has both: readiness and liveness on `/swagger.json`.

Metrics. The backend exposes Prometheus metrics at `/metrics` on the root of the service port, that is, outside `DCS_API_PATH`. The operator brings the scrape target for it; the chart does not ship a ServiceMonitor object usable for operations.

Log signals from which an operator can read the state:

Line	Meaning
bootstrap HTTP server listening on ... (initializing)	Process runs, initialization not yet through
waiting for PKCS#11 token (attempt N): ...	The token is not there yet. Logged immediately and then about once a minute; the attempt is retried every 5 seconds. The process does not die from this.
HTTP server listening on ...	Initialization complete
failed to sync federated catalogue schemas	Fatal; the instance restarts. <code>deploy.sh</code> searches for the prefix <code>failed to sync federated catalogue</code>
incomplete Federated Catalogue configuration; missing ...	Fatal; the message names the missing variables by name
federated catalogue readiness gate failed: ...	Fatal; typically a wrong catalogue context path
Could not read did document	<code>DCS_DID</code> is missing or points at nothing
did.json carries no usable keyAgreement method for dcs-ecdh	The published DID document and the HSM key set do not match

Scheduled jobs.

Job	Default schedule	Purpose
<fullname>-pac-monitor	every 5 minutes, <code>concurrencyPolicy: Forbid</code> , <code>startingDeadlineSeconds: 120</code>	Compliance pass. A missed run is not caught up; the next one derives the same state anew. History: 1 success, 3 failures.
<fullname>-acme	17 3 * * 1,4	Only with <code>acmeRenewal.enabled</code> . Twice a week; certbot only renews inside its window and otherwise exits 0

A failed compliance pass is a visible Kubernetes object, not a log line:

```
kubectl -n <ns> get jobs -l app.kubernetes.io/name=<release>-digital-contracting-service-pac-monitor
```

Retries. The backend retries failed PDF deliveries to peers and failed PDF regenerations on its own schedule. The production default is in the minute range; the test values lower it to 10s via `DCS_SYNC_FAIL_RETRY_INTERVAL` and `DCS_PDF_REGENERATION_RETRY_INTERVAL`.

Startup gates. These configurations are mandatory at startup; if one is missing, the instance does not come up: `DCS_PUBLIC_URL`, `DCS_PUBLIC_BASE_URL`, `DCS_DID`, `TSA_URL`, `PDF_CORE_URL` (the service must answer within three minutes), `IPFS_TENANT_BASE_URL` and `IPFS_MFS_BASE_URL`, `DCS_ISSUER_X5CHAIN_PATH` (the file must exist and end in a CA certificate), the Hydra variables (`HYDRA_PUBLIC_ISSUER_URL`, `HYDRA_CLIENT_ID`, `HYDRA_CLIENT_SECRET`, `HYDRA_REDIRECT_URI`, `HYDRA_ADMIN_URL`), a complete Federated Catalogue block (as soon as one of its values is set), plus a loadable trust document (`OID4VP_TRUST_DATA_PATH`).

A12.2 Recurring operational check

```
# 1. Nichts hÄngt
kubectl get pods -n <ns> --no-headers | grep -vE 'Running|Completed'

# 2. Die Instanz ist von auÄen auflÄsbar
for p in /.well-known/did.json /.well-known/dcs-agreement-credential.json /.well-known/dcs-federation-rules.md; do
  printf '%s ' "$p"; curl -s -o /dev/null -w '%{http_code}\n' "https://<host>$p"
done

# 3. Der Trust-Gate antwortet
kubectl -n <ns> exec deployment/<release>-digital-contracting-service -- \
  sh -c "curl -s -o /dev/null -w '%{http_code}' -X POST -H 'Content-Type: application/json' \
  -d '{\"peerDID\": \"did:web:probe\", \"direction\": \"inbound\"}' \"\${DCS_TRUST_PDP_URL}\""

# 4. Die Volumes tragen noch die vorgesehene Storage-Klasse
kubectl -n <ns> get pvc -o 'custom-columns=NAME:.metadata.name,CLASS:.spec.storageClassName,PHASE:.status.phase'

# 5. Die eigene Statusliste ist von auÄen abrufbar
curl -s -o /dev/null -w '%{http_code} %{content_type}\n' "https://<host>/status-list/1"
```

Expected: (1) no output, (2) 200 three times, (3) a 2xx code, (4) all claims `Bound` with the configured class, (5) 200 with the media type `application/statuslist+jwt`. If the fifth check does not return 200, all credentials of this instance are of unknown revocation status for every verifier, which reads like a revocation.

A13 Failure modes

All of the following cases have been observed on running deployments or follow necessarily from the artifacts. The symptom is in each case what the operator sees.

A13.1 Installation and startup

Symptom	Cause	Fix
Render aborts with the pointer to the dev trust fixture	<code>oid4vp.trust</code> points at the fixture inside the image without the release declaring <code>DCS_ALLOW_DEV_TRUST=true</code>	Create the trust document as a ConfigMap and reference it (A7)
Render aborts with <code>remote Federated Catalogue requires federatedCatalogue.remote.acknowledgeAdminAllTrustBoundary=true</code>	<code>federatedCatalogue.enabled</code> without <code>fcservice.enabled</code>	Acknowledge the trust boundary deliberately or run your own catalogue (A5.5)
Render aborts with <code>ingress.enabled=true</code> requires at least one entry in <code>ingress.hosts</code>	<code>ingress.hosts</code> is empty	Enter hostnames
Backend exits with <code>DCS_ISSUER_X5CHAIN_PATH</code> must point at this deployment's certificate chain	<code>pkcs11.provisioning.enabled=false</code> and <code>signing.issuerX5ChainPath</code> empty, or the path points at a file that is missing or does not end in a CA	Mount the chain into the backend container and set <code>signing.issuerX5ChainPath</code> (A6.3)
Backend pod <code>Running</code> but permanently <code>0/1 Ready</code> ; log repeats <code>waiting for PKCS#11 token (attempt N): CKR_GENERAL_ERROR</code>	The HSM provisioning job never started a pod. Occurs in particular when manifests were applied without Helm: an existing job gets patched instead of re-run	Delete the job and create it anew. The backend polls instead of crashing and recovers without a restart
Backend log <code>federated catalogue readiness gate failed: health check failed with status 404</code>	<code>federatedCatalogue.apiUrl</code> without the <code>fc-service</code> servlet context path. <code>fc-service</code> runs with <code>SERVER_SERVLET_CONTEXT_PATH=/fc</code> ; an in-cluster URL without <code>/fc</code> returns 404 on every call	Append the context path to the URL. Mind the asymmetry: a public URL like <code>https://<host>/fc</code> already contains it, an in-cluster URL <code>http://fc-service:8081</code> does not
Backend exits with <code>incomplete Federated Catalogue configuration; missing ...</code>	The catalogue block is all-or-nothing: as soon as one of the four variables is set, all four must be set	Add the values named in the message: <code>FEDERATED_CATALOGUE_API_URL</code> , <code>FEDERATED_CATALOGUE_CLIENT_ID</code> , <code>FEDERATED_CATALOGUE_CLIENT_SECRET</code> , <code>FC_KEYCLOAK_REALM_URL</code>
Backend exits with <code>Could not read did document</code>	<code>identity.enabled</code> is not set, so <code>DCS_DID</code> is not rendered	Set <code>identity.enabled: true</code> and make sure the identity source exists
Backend exits with <code>did.json carries no usable keyAgreement method for dcs-ecdh</code>	The published <code>did.json</code> comes from a source without a <code>keyAgreement</code> entry, or the token lacks the <code>dcs-ecdh</code> key with <code>CKA_DERIVE</code>	With SoftHSM2: re-run the provisioning job. With a real HSM: create the key with derive capability and regenerate <code>did.json</code>
<code>helm install</code> fails with <code>cannot create resource roles</code>	The installing identity may not create RBAC objects	Operate without RBAC permissions per A6.5

A13.2 Resources and scheduling

Symptom	Cause	Fix
A job sits at <code>Running</code> with zero pods; event: <code>Forbidden: minimum cpu usage per Container is 100m, but request is 50m</code>	A <code>LimitRange</code> rejects , it does not clamp up. The job controller keeps trying forever, which looks like a hang instead of a rejection	Read the actual <code>LimitRange</code> values (<code>kubectl -n <ns> describe limitrange</code>) and set the requests above them
Deployments exist with zero replicas, but no install error	The <code>ResourceQuota</code> 's <code>limits.cpu</code> is exhausted. A container without a <code>resources</code> block inherits the <code>LimitRange</code> default (often 1 CPU) and consumes the quota	Raise the quota, or set explicit small <code>resources.limits</code> . To size: <code>resources</code> , <code>pdfCore.resources</code> , <code>pkcs11.provisioning.resources</code> , <code>fcService.realmProvision.resources</code> , <code>fcService.resources</code> , plus <code>resources / jobResources</code> of the issuer releases
Pods stay <code>Pending</code> , PVC is <code>Pending</code>	No default <code>StorageClass</code> , or the configured class does not exist, or the requested size is below the CSI driver's minimum	Set the class. The HSM token deliberately requests 1 Gi instead of the ~10 MiB actually needed, because several drivers do not provision below that
TLS never gets issued, no error message	A <code>services</code> cap in the quota no longer admits cert-manager's ACME HTTP-01 solver; it needs its own service. Alternatively a <code>LimitRange</code> minimum prevents the solver pod from being scheduled	Raise the quota, or use the <code>CronJob</code> fallback <code>acmeRenewal</code> (A5.6)

A13.3 Network, ingress and federation

Symptom	Cause	Fix
<code>https://<host>/.well-known/did.json</code> returns 404, inside the cluster everything looks correct	Hydra's OIDC discovery claims the entire <code>/.well-known</code> prefix by default and shadows the three DCS documents	Route the three paths as exact rules ahead of the Hydra rule. Under ingress-nginx the chart does that by itself; under other controllers list them as full paths as in <code>values.bdd.yml</code>
A peer rejects the instance, nothing looks wrong locally	The served <code>did.json</code> carries a different <code>id</code> than <code>ISSUER_DID</code>	Align both and roll the backend out again. <code>deploy.sh --public-url</code> checks this
Federation transports nothing, no error message at all	The trust gate is fail-closed. <code>DCS_TRUST_PDP_URL</code> is not set or the endpoint does not answer with 2xx	Set <code>federation.trustPdpURL</code> and check reachability (A12.2). A reachable URL is not enough
Two instances do not federate although both are reachable	Both must publish <code>/.well-known/dcs-agreement-credential.json</code> , and the agreement compares a hash of the embedded federation rules. Two instances only federate on the same build	Bring both sides to the same revision
Token request at the catalogue Keycloak returns 401	<code>federatedCatalogue.oauth.clientID</code> names a client the realm does not create. The chart default <code>dcs-fc-client</code> does not exist in the bundled realm	Against the bundled catalogue set <code>federated-catalogue</code> ; against an external Keycloak the client created there
Catalogue calls fail across the board with 401	The realm URL <code>fc-service</code> checks the <code>iss</code> claim against is an in-cluster address, but the token was minted through the public one (or vice versa). Both sides must agree on the same address	Set the public realm URL in <code>fcKeycloak.realmURL</code> and keep <code>fcService.keycloak.hostname</code> in sync with <code>fcKeycloakApp.hostname</code> . That is manual work, because a <code>values</code> file cannot reference itself
<code>/.well-known/openid-configuration</code> of the catalogue Keycloak returns 200, but token issuance returns 500	The catalogue database was reset, Keycloak kept running and served realm metadata from memory over a schema that no longer existed	Set <code>fcService.realmProvision.enabled=true</code> and re-import the realm. Preventively: enable <code>fcPostgres.persistence.enabled</code> and <code>fcFuseki.persistence.enabled</code> , because the default is <code>emptyDir</code>
A second, old DCS release on the same host makes <code>/<pfad>/.well-known/did.json</code> answer 200 while the root returns 404	Two releases on one hostname. Fetching the wrong path reads like "instance is running"	One host, one DCS instance. Remove the old release including ingress, jobs, release secrets and PVCs

A13.4 Tooling and process problems

Symptom	Cause	Fix
Chart changes to a subchart have no effect	Helm prefers a packed <code>charts/*.tgz</code> over the source directory; an old archive serves old templates	<code>helm dependency build --skip-refresh deployment/helm</code> before every render. <code>deploy.sh</code> does this on its own
<code>helm list</code> shows nothing although objects sit in the namespace	The install was interrupted and is stuck at <code>pending-install</code> . Invisible to <code>helm list</code> , visible to <code>helm status</code>	Check <code>helm -n <ns> status <release></code> ; then clean up specifically or adopt with <code>helm upgrade --install</code>
A deployment produces nothing at all, without an error	Two concurrent operations on the same release. The lock applies per release name; different releases are unproblematic	Check <code>pgrep -af helm</code> before starting
An aborted deployment leaves an uncleaned release behind	A <code>helm</code> invocation was aborted from outside, for instance through a timeout of the calling tool. The child process survives a killed shell and interferes with the cleanup	Never wrap a deployment in a <code>timeout</code> and never start it in a tool invocation with a shorter deadline than Helm's own <code>--timeout</code>
<code>helm install</code> over a slow or unstable connection writes roughly one object every 20 minutes and never reports an error	The long-lived Helm client cannot handle the connection and retries with a long backoff. Recognizable by <code>helm status</code> , discovery and <code>helm install --dry-run=server</code> answering immediately while <code>kubectl apply</code> of the same manifests completes in seconds	Emergency route in A13.5. Check the connection first, because that route has a price
A <code>kubectl apply</code> onto an existing deployment fails with <code>may not specify more than 1 handler type</code>	Server-side apply merges probes. An old <code>exec</code> probe then sits next to the chart's <code>httpGet</code> probe	Remove the old and set the new in one JSON patch, or delete and recreate the deployment
A PVC stays in <code>Terminating</code>	<code>pvc-protection</code> holds the claim as long as any pod references it, including a <code>Succeeded</code> job pod	Delete the job pods, then the volume

A13.5 Emergency route: render with Helm, apply with kubectl

Only to be used when the Helm client cannot handle the connection to the cluster (A13.4) and the deployment cannot wait.

```
helm template <release> deployment/helm -n <ns> -f <ihre-werte>.yaml > /tmp/render.yaml
```

The rendered objects are then applied as ordinary objects. For a later `helm upgrade --install` to adopt them instead of colliding with them, they must carry the Helm ownership markers: the label `app.kubernetes.io/managed-by: Helm` plus the annotations `meta.helm.sh/release-name` and `meta.helm.sh/release-namespace`. `helm.sh/hook` annotations are removed in the process (otherwise Helm later treats the objects as hooks), and `helm.sh/hook: test` objects are skipped.

```
kubectl apply -n <ns> -f /tmp/objekte.yaml
```

The price, and it has to be known:

- **There is no Helm release record.** `helm list` shows nothing, `helm history` and `helm rollback` do not exist for this deployment. As soon as the connection carries again, adopt with `helm upgrade --install <release> deployment/helm -n <ns> -f <werte>.yaml`. The ownership markers above make that possible.
- **Helm's before-hook-creation deletion rule is lost.** Provisioning jobs get patched, not re-run. On a fresh installation that means: the HSM job never runs, the backend waits for its token forever. Delete the affected jobs before applying:

```
kubectl delete job -n <ns> <release>-digital-contracting-service-hsm-provision
kubectl delete job -n <ns> <release>-digital-contracting-service-fc-realm-provision
```

The reverse holds for an existing instance: do **not** delete and do **not** re-run the HSM job if the token volume is to be preserved. Otherwise there is the risk of a new instance DID, and every trust anchor pinned to it is invalid.

- **Existing jobs are immutable.** `apply` fails on them; delete them beforehand.
- A `pullPolicy: Always` only pulls at pod creation. After an `apply` that did not change the manifests, a rollout restart is needed:

```
kubectl rollout restart deploy -n <ns> <release>-digital-contracting-service \
<release>-digital-contracting-service-pdf-core <release>-orice
```

A14 Uninstall

A14.1 Read this first

`helm uninstall` deletes **all** PersistentVolumeClaims of this release. None of the PVCs in the chart carries a `helm.sh/resource-policy: keep` annotation. This makes the following disappear:

- the SoftHSM2 token and with it **the instance's identity**. A rebuilt deployment has a new DID; every trust document and every peer that knows the old DID is wrong afterwards;
- the IPFS repository and with it all artifact ciphertexts **including the audit trail content**;
- the Postgres volumes and with them the wrapped CEKs, without which even a backed-up IPFS repository is no longer readable, plus the allocation and revocation entries of the self-served status list (A5.6);
- the ORCE volume and with it the **archive notary evidence chain** (`/data/archive-audit-events.jsonl`, what the archive integrity check compares against) plus the serial number counter of the local TSA. On this volume sit records of evidence that cannot be reconstructed from the deployment.

Before any uninstall that does not concern a throwaway cluster, perform and verify the backup per `docs/backup-integration-guide.md`.

Not deleted along with it is the trust ConfigMap when it is referenced via `oid4vp.trust.existingConfigMap`. Helm does not manage it.

A14.2 Procedure

Prerequisites: Backup exists and is verified; peers are informed about the withdrawal if the instance was federated.

Steps:

1. Remove the release:

```
helm uninstall <release> --namespace <ns>
```

2. Check remaining objects and remove them specifically. Jobs still holding a PVC block its deletion:

```
kubectl -n <ns> get pods,jobs,pvc,ingress -l app.kubernetes.io/instance=<release>
```

3. Issuer releases of the same namespace, if they are to go as well:

```
helm uninstall dcs-issuer --namespace <ns>
```

With the issuer volume, its root CA, keys and revocation bits disappear. A newly installed issuer mints a new root; every anchor bundle and every issued credential naming the old status list is invalid afterwards (A7.2).

4. Objects not managed by Helm: the trust ConfigMap, pre-created secrets and manually created ingress rules.

Verification:

```
helm -n <ns> list
kubectl -n <ns> get all,pvc
```

The first output must no longer contain the release, the second no objects with the release label.

Failure modes:

Symptom	Cause	Fix
A PVC stays in <code>Terminating</code>	<code>pvc-protection</code> : a pod still references it, including a <code>Succeeded</code> job pod	Delete the job pods, then the volume
Ingress rules keep answering after the uninstall	Manually created rules or rules from another release	Find them via <code>kubectl -n <ns> get ingress</code> and remove them individually

A14.3 `uninstall.sh` is not the operator route

`deployment/helm/uninstall.sh` belongs to the Node-RED deployment package under `deployment/node-red/`, which creates and tears down DCS instances from within a flow. It is called from there with `<kubeconfig> <instancename>` and derives from that a fixed namespace `digital-contracting-service-<instancename>` and release `digital-contracting-service`. It then **deletes the entire namespace**.

Against a deployment installed per A8 this naming scheme does not fit, and the namespace deletion also hits objects that do not belong to the release. For an instance from this guide, A14.2 applies.

Part B: Local development and test setup

This part describes tools that have no place in production: locally running processes without containers, key fixtures living in the repository, and test credentials. They are acceptable here because a development stack carries no confidentiality. None of the values files from this part may be used in a reachable cluster.

B1 Prerequisites

- Rancher Desktop (or an equivalent local Kubernetes environment) with NodePort forwarding to `localhost`
- Go 1.25+ with `air` (`backend/go.mod` requires `go 1.25.7`)
- Node.js 22+ (build image and CI use 22)
- Python 3.10+, `Goa v3`, `make`, `curl`, `OpenSSL`
- SoftHSM2 with `libsofthsm2.so` and `softhsm2-util`, plus OpenSC for `pkcs11-tool`. The development keys live in a PKCS#11 token, not in files.
- `helm`, `kubectl`, `docker`
- For the BDD suite additionally `kind` (CI uses v0.23.0), `git` (the runner clones the `bdd-executor` if it does not sit next to the repository), plus `nc` and `fuser` (the runner uses them to check and clear its local ports)

B2 Complete stack with one command

Prerequisite: a SoftHSM token must exist. The backend opens a PKCS#11 token at startup and does not get past bootstrap without one. There is no software fallback, not even locally. `dev-stack.sh` creates the token itself in step 8 (`$HOME/.dcs/softhsm-8991`, label `dcs`) and is idempotent doing so: an existing token and existing keys stay untouched. This requires `softhsm2-util` and `pkcs11-tool` to be installed (B1); if they are missing, the provisioning script tries to install them via `apt-get` and needs `sudo` for that.

```
bash dev-stack.sh
```

The script, in this order:

1. `helm dependency build --skip-refresh deployment/helm`, then `helm upgrade --install dcs deployment/helm -f deployment/helm/values.dev.yml`
2. installs the credential issuer as its own release `dcs-issuer` from `deployment/helm/charts/orce` with `values.issuer-base.yml` and `values.issuer.dev.yml` (A7) and waits for its rollout. It is reached via `ISSUER_BASE_URL` (default `http://localhost:30181`); that must be the address under which the **backend** reaches the issuer, not merely one that works: every issued credential names `<ISSUER_BASE_URL>/status-list/1`, and the verifier rejects a list whose `sub` does not match this URI
3. waits for the Federated Catalogue if the values file enables it. The backend's catalogue readiness gate makes exactly one attempt; a backend process started too early dies from it
4. exports the TSA certificate from the secret `dcs-orce-tsa-material` to `backend/certs/dev/orce-tsa-cert.pem`
5. waits for the release's status list service. Nothing reads it (A5.6); the step remains until the subchart is removed
6. installs the testWallet venv (`make -C testWallet install`) and checks with `make -C testWallet check-status-list` that the status list served by the issuer is usable: it answers at the URI the committed credentials name, its chain ends, compared by fingerprint, in a committed anchor, and its leaf names the issuer. Each of these failures would later read like a revoked credential
7. copies `backend/.env.dev1` to `backend/.env`
8. provisions the local SoftHSM2 token under `$HOME/.dcs/softhsm-8991` with the five instance keys and generates `backend/certs/dev/did-8991.json` with `gendid`
9. issues the C2PA x5chain, with CRL address, origin `http://localhost:8991` and `did:web:localhost%3A8991` as SANs in the leaf, and publishes an empty CRL. `DCS_ISSUER_X5CHAIN_PATH` in `.env.dev1` points at this chain; with it the backend signs its own status list and the C2PA manifests
10. starts `pdf-core` with `air`, waits for `http://localhost:8080/version`
11. starts the EU DSS demo image at `http://localhost:18099` (can be disabled with `DCS_DEV_DSS=0`)
12. starts the Vite dev server, generates the Goa layer (`goa gen`) and starts the backend with `air`

Backend: `http://localhost:8991`. Vite: `http://localhost:5173`, proxies `/api` to the backend. Stop with `Ctrl+C` in the same terminal.

`values.dev.yml` sets `replicaCount: 0` and `pdfCore.enabled: false`. Backend and pdf-core run natively instead of in the cluster, which is why no in-cluster token gets provisioned either; the token lives on the development machine.

The exposed NodePorts are listed at the top of `values.dev.yml` (instance A) and `values.dev2.yml` (instance B); the two instances occupy disjoint port ranges.

Verification: `curl http://localhost:8991/api/...` answers and `/tmp/backend-live.log` contains `HTTP server listening`. That the token is in place is additionally shown by:

```
SOFTHSM2_CONF=$HOME/.dcs/softhsm-8991/softhsm2.conf softhsm2-util --show-slots | grep -A2 'Label:'
```

A slot with the label `dcs` must appear. If one of the three processes exits, `dev-stack.sh` exits with its status; for backend and pdf-core it first prints the last 200 log lines of the affected process.

Anyone starting the stack by hand instead of via `dev-stack.sh` must keep the order. The backend merely waits for a missing token (it polls, see A12.1); the other three dependencies are hard startup gates and otherwise produce a process that dies right away:

1. The SoftHSM token must exist before the backend starts. By hand: `bash scripts/hsm-provision.sh $HOME/.dcs/softhsm-8991 dcs 1234 12345678`.
2. The C2PA x5chain must exist before the backend starts (`DCS_ISSUER_X5CHAIN_PATH` in `.env.dev1`). By hand: the `c2pa-cert-provision.sh` call from step 9 above. If the file is missing, the process exits immediately.
3. The Federated Catalogue must answer before the backend starts. The readiness gate makes exactly **one** attempt. That is intentional, so that a broken catalogue is not masked by retries.
4. **pdf-core must be running** (`http://localhost:8080`). The backend tries `/version` for up to three minutes and then exits. A manual sequence that starts only backend and frontend does not work.

Item 4 is the one most often overlooked. pdf-core is its own process (`cd pdf-core && air`), not part of the backend.

Second instance. `bash dev-stack2.sh` starts instance B on `:8992` with the frontend on `:5174` and its own token under `~/.dcs/softhsm-8992`. It **shares pdf-core and the credential issuer with instance A** and checks both at startup; instance A must therefore be running first. The script currently has one gap: it provisions **no** C2PA x5chain for instance B, although `backend/.env.dev2` expects it at `../pdf-core/certs/dev/c2pa-x5chain-8992.pem`. Without the file, backend B exits immediately. Until the script catches up, issue it once by hand, with the same arguments as for instance A, only with the `-8992` token, `http://localhost:8992` and `did:web:localhost%3A8992`:

```
bash scripts/c2pa-cert-provision.sh "$HOME/.dcs/softhsm-8992" dcs 1234 \
pdf-core/certs/dev/c2pa-x5chain-8992.pem \
/usr/lib/softhsm/libsofthsm2.so \
"http://localhost:8992/crl/dcs-c2pa.crl" \
"http://localhost:8992" "did:web:localhost%3A8992"
```

B3 BDD and E2E suites

All targets live in `tests/bdd/Makefile`.

```
# Cluster aufbauen, beide Instanzen deployen, Umgebung vorbereiten
make -C tests/bdd kind_up

# Suite gegen den stehenden Cluster ausführen (beliebig oft)
make -C tests/bdd run_bdd_kind_once
make -C tests/bdd run_bdd_kind_once F=features/<pfad>

# Cluster abbauen und Artefakte aufräumen
make -C tests/bdd kind_down
```

One-shot full cycle, as CI runs it:

```
make -C tests/bdd run_bdd_kind_ci
```

Further targets: `run_e2e_kind_ci` (Playwright instead of behave), `run_bdd_audit_kind_ci` and `kind_up_audit` / `run_bdd_audit_kind_once` for the isolated audit gate with only one instance, `kind_delete` for plain cluster deletion. `make help` lists the common ones.

Two targets must not be used: `run_bdd_kind_all` calls a `run_bdd_helm_all` that the Makefile does not define; the call aborts with `No rule to make target`. And a `run_bdd_helm_dev` does not exist. Against an already running release, `run_bdd_helm` is the right target (B4.2).

Calling Playwright directly

Anyone running the E2E suite against an existing release instead of starting it via `run_e2e_kind_ci` does not get the port forwards delivered: the runner sets them up at start and tears them down at the end. Without the forward to DSS, every spec that signs aborts, because the wallet expects its external signing component at `localhost:18099`. So set them up yourself before the run:

```
kubect1 -n <ns> port-forward svc/<release>-dss      18099:8080 &
kubect1 -n <ns> port-forward svc/<release>-orce     18880:1880 &
kubect1 -n <ns> port-forward svc/<release>-postgresql 5432:5432 &
```

A forward is attached to the pod, not to the service. A rollout ends it, and the next signing run then fails with `ECONNREFUSED` on 18099.

The specs create their role logins through a Python subprocess. Its path is in `E2E_BDD_PYTHON`; the default is the venv the BDD Makefile creates. If Playwright is driven from a different interpreter than the one owning this venv, the variable must be set, otherwise the very first login fails.

`run_bdd_helm` checks before the test run that the stack really answers, and otherwise aborts with a diagnosis. The wait conditions are proven startup signals; anyone checking the stack by hand can follow the same order:

1. Deployment and backend pod of instance A are available and `ready` (180 s deadline each); if that fails, a Federated Catalogue diagnosis follows.
2. The login endpoint answers **through the Traefik ingress** on port 18080. A Go standard 404 response is recognized as its own failure mode: the host port does not reach the ingress.
3. Instance B, if present: deployment available (300 s) and HTTP via `dcx-b.localhost` (up to 120 s). If it is missing, the runner warns explicitly instead of passing silently; the `@two-instance` scenarios then fail.
4. The DSS deployment is available (deadline 420 s; the Tomcat bundle boots slowly, its readiness probe only starts after 90 s).
5. ORCE is available, the archive audit token is readable from the running pod, the audit log endpoint answers, and the contract target flow does not answer 404 through the ingress.
6. The status list served by the issuer release is usable (`ISSUER_BASE_URL`, default `<origin>/issuer`): it answers at the URI the suite credentials name, its chain ends by fingerprint in a committed anchor, its leaf names the issuer, and it is large enough for the allocated indices. Each of these failures would show up in the suite as a revoked credential, which is why it is named up front.

After that the runner sets up port forwards for **PostgreSQL (5432), the DCS service (18991), DSS (18099) and ORCE (18880)**. The DCS forward serves the endpoints the ingress does not route, such as `/metrics`. Keycloak is not forwarded. Beforehand the runner kills every process occupying ports 5432, 18991 or 18880; a locally running PostgreSQL on 5432 is killed along with them. The standard run excludes `@isolated_stack` scenarios, they run only in their own targets. A run that executed zero scenarios counts as a failure, not as green.

The flow of `kind_deploy` in short: create or reuse the kind cluster, load the pre-built DCS and pdf-core images plus the DSS image into it, update Helm repositories and build dependencies, apply the Prometheus stack CRDs, create the namespace `dcx-bdd`, prepare the ORCE volume, determine the host address for the checkpoint sink, `helm upgrade --install dcs` with `values.bdd.yml` (public origin, Hydra and checkpoint sink addresses are added via `--set`), CoreDNS rewrite for `dcx-a.localhost` / `dcx-b.localhost`, then the issuer release `dcx-issuer` from `values.issuer-base.yml` + `values.issuer.bdd.yml`. The issuer comes **after** the DCS release because its ingress references a Traefik middleware and the Traefik CRDs only arrive with that release; the trust anchors here are committed fixtures, so unlike in production (A7.2) nothing depends on the reverse order. Then a rollout restart of the freshly built workloads and waiting for the rollout. `kind_deploy_b` afterwards puts instance B with `values.bdd2.yml` into the same namespace, strictly after, because B uses several services provided by A (Traefik, ORCE including TSA and trust PDP, Federated Catalogue, DSS, IPFS) and actively queries some of them during its own startup; B's status list sidecar moreover checks its readiness through A's Traefik against the status list service of the A release.

The HTTPS sink for the checkpoint scenarios is run by the suite itself: a listener in the behave process on host port 18443 (`BDD_CHECKPOINT_SINK_PORT`). `kind_deploy` sets `orcx.checkpointSink.url` to an address of this host reachable from the cluster (`host.docker.internal`, under WSL the Rancher Desktop veth address) and aborts if none can be determined. Port 18443 must be free on the host.

The deployment deliberately runs **without** `--wait`: Helm holds back `post-install` hooks until all deployments are ready, and the backend only becomes ready once the HSM hook has generated its material. That is why Helm returns immediately, and the Makefile afterwards waits separately for the rollout.

B4 Deviations of the local cluster variant

`values.bdd.yml` is tailored to kind. Against a different local cluster, two deviations apply.

B4.1 Rancher Desktop k3s instead of kind: Traefik hostPort

Symptom. The Traefik pod stays `Pending`. As a consequence, the status list sidecar of **both** DCS pods never becomes `ready`, because its readiness probe requires a real HTTP route through the ingress. The backend pod thereby remains permanently not ready although the backend container itself is fine.

Cause. `values.bdd.yml` sets `traefik.ports.web.hostPort: 18080` for kind. On k3s, Klipper's `svc1b` already occupies this port, the pod finds no node and stays unscheduled.

Fix. Neutralize the hostPort at deployment time:

```
helm upgrade --install dcs deployment/helm -n dcs-bdd \
  -f deployment/helm/values.bdd.yml \
  --set traefik.ports.web.hostPort=null
```

Verification.

```
kubectl -n kube-system get pod -l app.kubernetes.io/name=traefik
kubectl -n dcs-bdd get pods -o 'custom-
columns=POD:.metadata.name,CONTAINER:.status.containerStatuses[*].name,READY:.status.containerStatuses[*].ready'
```

The Traefik pod must be `Running`, and in the DCS pod **both** containers must report `true`.

B4.2 KUBECONFIG binding of the `kind_*` targets

Every `kind_*` target in the Makefile exports `KUBECONFIG` to `tests/bdd/.tmp/kind-kubeconfig`. That is intentional: otherwise deployments land wherever the developer's current context points, while the suite's data path (port 18080) always reaches the kind cluster.

The consequence: **none of these targets may be used against a cluster other than kind**. There, call `make run_bdd_helm` directly. It does not belong to the `kind_*` group and respects the existing `KUBECONFIG`:

```
make -C tests/bdd run_bdd_helm K8S_NAMESPACE=<ns> HELM_RELEASE=<release>
```

Further overridable variables with defaults: `K8S_NAMESPACE` (`default`), `HELM_RELEASE` (`dcs`), `FEATURES_PATH` (`features`), `BDD_INGRESS_PORT` (`18080`), `LOCAL_FORWARD_PORT` (`18991`), `RUN_MODE` (`bdd`, alternatively `e2e`), `HELM_TIMEOUT` (`20m`).

Under WSL the Makefile places the Python venv under `$HOME/.dcs-bdd-venv` by default instead of inside the repository. A venv on the 9P filesystem slows every Python import considerably. `VENV_PATH=<pfad>` overrides this.

B5 What from part B does not belong in production

Mechanism	Why it exists locally	Why it is inadmissible in production
<code>DCS_ALLOW_DEV_TRUST=true</code>	Permits the trust fixture inside the image	Its issuer keys live in the repository; anyone with a copy can mint accepted credentials
<code>pkiRootCA.devFixture</code> (issuer overlays for dev and BDD)	Hands the issuer the committed root so that it matches the equally committed anchor bundles	The private key of this root lives in the repository; the backend accepts it as an anchor only under <code>DCS_ALLOW_DEV_TRUST</code>
<code>JWT_ALG_NONE_SUPPORTED</code> , <code>AUTH_INSECURE_COOKIES</code>	Simplify the test setup	Both remove security properties
<code>image.pullPolicy: Never</code>	Uses images loaded locally into kind	No pod starts in a cluster without these images
<code>hydra.config.dev: true</code>	Permits <code>http</code> issuer URLs	Permits unencrypted OIDC endpoints
Hard-wired test credentials in the <code>values.dev*</code> / <code>values.bdd*</code> files (Hydra clients, catalogue secret, archive token)	Reproducible test runs	Publicly known values
<code>statusListLocalhostProxy</code>	Resolves a <code>localhost</code> lookup inside the test cluster	A sidecar without purpose outside this setup
<code>serviceMonitor.enabled</code>	Attaches the test instance to the kind cluster's Prometheus	The rendered object hard-codes the namespace <code>dcs-bdd</code> and the selector <code>release: prometheus</code> . In any other namespace it is ineffective or rejected. The scrape target for operations is created by the operator (A12.1)
SoftHSM2 in the cluster	No HSM needed	A software token; the private keys sit on a cluster volume
<code>orce.syntheticPeer.enabled</code>	Synthetic peer identities for federation scenarios	Test scaffolding; additionally consumes services against a namespace quota
<code>postgresql.persistence.enabled: false</code> (chart default)	Fast throwaway clusters	A pod restart loses every contract